

410 10. Học sâu

- Nếu ta sử dụng K bộ lọc tích chập khác nhau ở lớp ẩn đầu tiên này, ta sẽ thu được K bản đồ đặc trưng đầu ra hai chiều, được coi chung như một bản đồ đặc trưng ba chiều duy nhất. Ta xem mỗi trong số K bản đồ đặc trưng đầu ra như một kênh thông tin riêng biệt, vì vậy giờ đây ta có K kênh trái ngược với ba kênh màu của bản đồ đặc trưng đầu vào ban đầu. Bản đồ đặc trưng ba chiều giống như các kích hoạt trong một lớp ẩn của mạng nơ-ron đơn giản, ngoại trừ việc chúng được tổ chức và tạo ra theo cấu trúc không gian.
- Chúng tôi thường áp dụng hàm kích hoạt ReLU (10.5) cho hình ảnh tích chập. Bước này đôi khi được xem như một lớp riêng biệt trong mạng nơ-ron tích chập, trong trường hợp đó nó được gọi là lớp phát hiện.

lớp phát
hiện

10.3.2 Lớp gộp

Lớp gộp (pooling layer) cung cấp một cách để thu gọn một hình ảnh lớn thành một hình ảnh tóm tắt nhỏ hơn. Mặc dù có một số cách khả thi để thực hiện gộp, nhưng phép toán gộp tối đa (max pooling) tóm tắt mỗi khối pixel 2 × 2 không chồng chéo trong hình ảnh bằng cách sử dụng giá trị lớn nhất trong khối đó. Điều này làm giảm kích thước của hình ảnh đi một nửa theo mỗi hướng, và nó cũng cung cấp một số tính bất biến vị trí: tức là miễn là có một giá trị lớn trong một trong bốn pixel trong khối, toàn bộ khối đó được coi là một giá trị lớn trong hình ảnh đã được thu gọn.

gộp chung

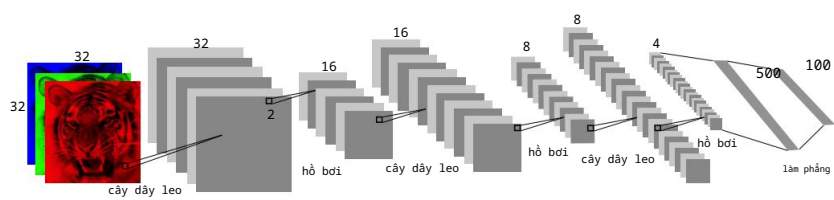
Dưới đây là một ví dụ đơn giản về max pooling:

	1253	
	3012	35
Max pool	2134	24
	1120	

10.3.3 Kiến trúc của mạng nơ-ron tích chập

Cho đến nay, chúng ta đã định nghĩa một lớp tích chập duy nhất – mỗi bộ lọc tạo ra một bản đồ đặc trưng hai chiều mới. Số lượng bộ lọc tích chập trong một lớp tích chập tương đương với số lượng đơn vị tại một lớp ẩn cụ thể trong mạng nơ-ron kết nối đầy đủ kiểu như chúng ta đã thấy trong Phần 10.2. Con số này cũng xác định số lượng kênh trong bản đồ đặc trưng ba chiều thu được. Chúng tôi cũng đã mô tả một lớp gộp (pooling layer), giúp giảm hai chiều đầu tiên của mỗi bản đồ đặc trưng ba chiều. Mạng CNN sâu có rất nhiều lớp như vậy. Hình 10.8 thể hiện kiến trúc điển hình của một mạng CNN cho nhiệm vụ phân loại ảnh CIFAR100 .

Ở lớp đầu vào, ta thấy bản đồ đặc trưng ba chiều của một ảnh màu, trong đó trục kênh biểu diễn mỗi màu bằng một bản đồ đặc trưng hai chiều 32 × 32 pixel. Mỗi bộ lọc tích chập tạo ra một kênh mới ở lớp ẩn đầu tiên, mỗi kênh là một bản đồ đặc trưng 32 × 32 (sau khi thêm một số phần đệm ở các cạnh). Sau vòng tích chập đầu tiên này, ta có một "ảnh" mới; một bản đồ đặc trưng với số kênh nhiều hơn đáng kể so với ba kênh màu đầu vào (sáu kênh trong hình, vì ta đã sử dụng sáu bộ lọc tích chập).



HÌNH 10.8. Kiến trúc của một mạng CNN sâu cho nhiệm vụ phân loại CIFAR100 . Các lớp tích chập được xen kẽ với các lớp gộp tối đa 2×2 , giúp giảm kích thước xuống một nửa ở cả hai chiều.

Tiếp theo là lớp max-pool, giúp giảm kích thước của đối tượng. Bản đồ trong mỗi kênh được nhân với hệ số bốn: hai trong mỗi chiều.

Trình tự tích chập rồi gộp này được lặp lại cho hai lớp tiếp theo. Một số chi tiết như sau:

- Mỗi lớp tích chập tiếp theo đều tương tự như lớp đầu tiên. Nó nhận đầu vào là bản đồ đặc trưng ba chiều từ lớp trước đó và xử lý nó như một hình ảnh đa kênh duy nhất. Mỗi bộ lọc tích chập được học có số kênh bằng với số kênh của bản đồ đặc trưng này.
- Vì kích thước của các bản đồ đặc trưng kênh bị giảm sau mỗi lớp gộp, chúng ta thường tăng số lượng bộ lọc trong lớp tích chập tiếp theo để bù lại.
- Đôi khi chúng ta lặp lại một vài lớp tích chập trước lớp gộp. Điều này giúp tăng kích thước của bộ lọc một cách hiệu quả.

Các thao tác này được lặp lại cho đến khi quá trình gộp giảm mỗi bản đồ đặc trưng kênh xuống chỉ còn một vài pixel ở mỗi chiều. Tại thời điểm này, các bản đồ đặc trưng ba chiều được làm phẳng – các pixel được coi là các đơn vị riêng biệt – và được đưa vào một hoặc nhiều lớp kết nối đầy đủ trước khi đến lớp đầu ra, đó là hàm kích hoạt softmax cho 100 lớp (như trong (10.13)).

Có rất nhiều tham số điều chỉnh cần được lựa chọn khi xây dựng một mạng nơ-ron tích chập, ngoài số lượng, bản chất và kích thước của mỗi lớp. Có thể sử dụng kỹ thuật học bỏ qua (dropout learning) ở mỗi lớp, cũng như các phương pháp điều chỉnh lasso hoặc ridge (xem Mục 10.7). Chi tiết về việc xây dựng một mạng nơ-ron tích chập có vẻ khá phức tạp. May mắn thay, có nhiều phần mềm tuyệt vời với các ví dụ và tài liệu hướng dẫn chi tiết, giúp người dùng lựa chọn các tham số một cách hợp lý. Đối với tập dữ liệu kiểm thử chính thức CIFAR100 , độ chính xác tốt nhất tính đến thời điểm viết bài này là hơn 75%, nhưng chắc chắn hiệu suất này sẽ tiếp tục được cải thiện.

10.3.4 Tăng cường dữ liệu

Một thủ thuật quan trọng khác được sử dụng trong mô hình hóa hình ảnh là tăng cường dữ liệu. Về cơ bản, mỗi hình ảnh huấn luyện được sao chép nhiều lần, mỗi bản sao được biến dạng ngẫu nhiên theo cách tự nhiên sao cho khả năng nhận dạng của con người không bị ảnh hưởng. Hình 10.9 hiển thị một số ví dụ. Các biến dạng điển hình là

dữ liệu tháng 8-
tâm trí



HÌNH 10.9. Tăng cường dữ liệu. Hình ảnh gốc (bên trái) bị biến dạng theo những cách tự nhiên để tạo ra các hình ảnh khác nhau với cùng nhãn lớp. Những hình ảnh này Các biến dạng không đánh lừa được con người và hoạt động như một hình thức chuẩn hóa khi khớp dữ liệu. CNN.

thu phóng, dịch chuyển ngang và dọc, biến dạng, xoay nhỏ, và trong trường hợp này lật ngang. Thoạt nhìn, đây là một cách để tăng số lượng dữ liệu huấn luyện đáng kể với những ví dụ hơi khác nhau, và do đó bảo vệ chống lại

Hình tương quá khớp (overfitting). Trên thực tế, chúng ta có thể xem đây như một hình thức điều chỉnh (regularization): chúng ta xây dựng một

Một đám mây hình ảnh bao quanh mỗi hình ảnh gốc, tất cả đều có cùng nhãn. Điều này

Việc làm dày dữ liệu theo kiểu này có tính thần tượng tự như phương pháp điều chỉnh theo đường gờ (ridge regularization).

Chúng ta sẽ thấy trong Phần 10.7.2 rằng các thuật toán giảm độ dốc ngẫu nhiên để huấn luyện các mô hình học sâu xử lý lặp đi lặp lại các lô ảnh được chọn ngẫu nhiên, ví dụ như 128 ảnh huấn luyện một lúc. Điều này hoạt động rất hiệu quả với việc tăng cường dữ liệu, bởi vì chúng ta có thể làm biến dạng mỗi ảnh trong lô ảnh đó.

Ngay lập tức, và do đó không cần phải lưu trữ tất cả các hình ảnh mới.

10.3.5 Kết quả sử dụng bộ phân loại được huấn luyện trước

Ở đây, chúng tôi sử dụng bộ phân loại được huấn luyện trước ở cấp độ ngành để dự đoán lớp của Một số hình ảnh mới. Bộ phân loại **ResNet50** là một mạng nơ-ron tích chập. Mô hình này được huấn luyện bằng cách sử dụng tập dữ liệu **ImageNet** , bao gồm hàng triệu hình ảnh. hình ảnh thuộc về số lượng danh mục ngày càng tăng.10 Hình 10.10

Hình ảnh minh họa hiệu suất của **resnet50** trên sáu bức ảnh (bộ sưu tập cá nhân của một trong các tác giả).¹¹ Mạng CNN thực hiện việc phân loại khá tốt. con điều hầu trong hình thứ hai. Nếu ta thu nhỏ lại như trong hình thứ ba, thì nó...

Bị nhầm lẫn và chọn đài phun nước thay vì con chim ưng. Trong phần cuối cùng Hình ảnh "chim jacamar" là một loài chim nhiệt đới đến từ Nam và Trung Mỹ.

Màu sắc tương tự như loài chim Cape Weaver của Nam Phi. Chúng tôi sẽ cung cấp thêm chi tiết.

Ví dụ này được trình bày trong Mục 10.9.4.

Phần lớn công việc trong việc huấn luyện mạng CNN nằm ở việc học các bộ lọc tích chập.

ở các lớp ẩn; đây là các hệ số của mạng CNN. Đối với các mô hình được điều chỉnh cho

các kho dữ liệu khổng lồ như **ImageNet** với nhiều lớp, kết quả đầu ra của những điều này

Các bộ lọc có thể đóng vai trò là các đặc trưng cho các bài toán phân loại ảnh tự nhiên nói chung. Người ta có thể sử dụng các lớp ẩn được huấn luyện trước này cho các bài toán mới với

sử dụng các tập dữ liệu huấn luyện nhỏ hơn nhiều (một quá trình được gọi là đóng băng trọng số), và chỉ huấn luyện một vài lớp cuối cùng của mạng, điều này đòi hỏi ít dữ liệu hơn nhiều.

cân nặng
đóng băng

10Để biết thêm thông tin về **resnet50**, hãy xem He, Zhang, Ren và Sun (2015) "Deep "Học tập dư thừa cho nhận dạng hình ảnh", <https://arxiv.org/abs/1512.03385>. Để biết thêm chi tiết về **ImageNet**, xem Russakovsky, Deng, et al. (2015) "ImageNet Large Scale "Thử thách nhận dạng hình ảnh", trong Tạp chí Quốc tế về Thị giác Máy tính.

11. Kết quả của **ResNet** có thể thay đổi theo thời gian, vì mô hình được huấn luyện công khai sẽ được cập nhật thường xuyên. Cập nhật định kỳ.





hồng hạc	Chim ưng Cooper	Chim ưng Cooper
hồng hạc 0,83 điều	Đài phun nước 0,60 0,35	
mỏ thìa cò hâu 0,17 cú xám lớn 0,00	0,09 đỉnh 0,12	
trắng chim sẻ	Móc 0,06 0,07	

Chó Lhasa Apso Cape weaver	con mèo
Chó sục Tây Tạng 0,56 Chó chăn cừu Anh cổ 0,82 Chó Jacamar 0,28	
Lhasa 0,32 Shih-Tzu 0,04 vẹt đuôi dài 0,12	
Chó Cocker Spaniel 0,03 Mèo Ba Tư 0,04 Chim Robin 0,12	

HÌNH 10.10. Phân loại sáu bức ảnh bằng mạng nơ-ron tích chập `resnet50` .
được huấn luyện trên tập dữ liệu `ImageNet` . Bảng bên dưới hình ảnh hiển thị kết quả thực tế.
(Dự định) nhấn ở đầu mỗi bảng và ba lựa chọn hàng đầu của bộ phân loại.
(trên thang điểm 100). Các con số là xác suất ước tính cho mỗi lựa chọn. (Một con điều)
(Là một loài chim săn mồi, nhưng không phải là điều hâu.)

Các đoạn văn ngắn và sách 12 đi kèm với gói `keras` cung cấp thêm thông tin.
Thông tin chi tiết về các ứng dụng đó.

10.4 Phân loại tài liệu

Trong phần này, chúng tôi giới thiệu một loại ví dụ mới có ý nghĩa quan trọng.
Ứng dụng trong công nghiệp và khoa học: dự đoán thuộc tính của tài liệu.
Ví dụ về các tài liệu bao gồm các bài báo trên tạp chí y khoa, tin tức của Reuters.
nguồn cấp dữ liệu, email, tweet, v.v. Ví dụ của chúng ta sẽ là `IMDb` (Internet Movie Board)
(Cơ sở dữ liệu) xếp hạng – các tài liệu ngắn ghi lại những lời phê bình của người xem
của phim.¹³ Phản hồi trong trường hợp này là **cảm nhận** của bài đánh giá, mà
Sẽ là kết quả tích cực hoặc tiêu cực.

12. Học sâu với R, của F. Chollet và J.J. Allaire, 2018, Nhà xuất bản Manning.
13Để biết thêm chi tiết, xem Maas et al. (2011) “Học các vectơ từ để phân tích cảm xúc”,
Trong Kỷ yếu Hội nghị thường niên lần thứ 49 của Hiệp hội Ngôn ngữ học tính toán: Công nghệ
ngôn ngữ con người, trang 142-150.

414 10. Học sâu

Đây là phần mở đầu của một bài đánh giá tiêu cực khá hài hước:

Đây chắc chắn là một trong những bộ phim tệ nhất thập niên 1990. Khi tôi và bạn bè xem phim này (với tư cách là đối tượng khán giả mà phim nhắm đến), chúng tôi chỉ ngồi xem nửa tiếng đầu với vẻ mặt kinh ngạc vì độ tệ hại của nó. Phần thời gian còn lại, mọi người trong rạp bắt đầu nói chuyện với nhau, bỏ đi hoặc khóc lóc úp mặt vào bóng ghế .

Mỗi bài đánh giá có thể có độ dài khác nhau, chứa tiếng lóng hoặc từ ngữ không có nghĩa, có lỗi chính tả, v.v. Chúng ta cần tìm cách để làm nổi bật loại tài liệu này.

tạo tính năng

Đây là cách nói hiện đại để chỉ việc xác định một tập hợp các biến dự báo.

Phương pháp trích xuất đặc trưng đơn giản và phổ biến nhất là mô hình túi từ (bag-of-words). Chúng tôi chấm điểm cho mỗi tài liệu dựa trên sự hiện diện hoặc vắng mặt của từng từ trong từ điển ngôn ngữ – trong trường hợp này là từ điển tiếng Anh. Nếu từ điển chứa M từ, điều đó có nghĩa là với mỗi tài liệu, chúng tôi tạo ra một vectơ đặc trưng nhị phân có độ dài M, và chấm điểm 1 cho mỗi từ có mặt, và 0 nếu không có từ nào xuất hiện.

túi từ

Đó có thể là một vectơ đặc trưng rất rộng, vì vậy chúng tôi giới hạn từ điển – trong trường hợp này là 10.000 từ xuất hiện thường xuyên nhất trong tập dữ liệu huấn luyện gồm 25.000 bài đánh giá. May mắn thay, có những công cụ tuyệt vời để thực hiện việc này một cách tự động. Đây là phần đầu của một bài đánh giá tích cực đã được biên tập theo cách này:

BẮT ĐẦU bộ phim này thật sự xuất sắc, dàn diễn viên, địa điểm quay phim, phong cảnh, cốt truyện, đạo diễn, mọi người đều rất phù hợp với vai diễn của mình và bạn có thể tưởng tượng mình đang ở đó. Robert UNK là một diễn viên tuyệt vời và giờ đây cũng là một đạo diễn tài ba. Cha của UNK đến từ cùng một hòn đảo ở Scotland với tôi nên tôi rất thích ...

Ở đây, ta có thể thấy nhiều từ đã bị lược bỏ, và một số từ không xác định (UNK) đã được đánh dấu như vậy. Với sự giảm bớt này, vectơ đặc trưng nhị phân có độ dài 10.000, và chủ yếu bao gồm các số 0 và một số ít số 1 ở các vị trí tương ứng với các từ có trong tài liệu.

Chúng ta có một tập dữ liệu huấn luyện và một tập dữ liệu kiểm tra, mỗi tập gồm 25.000 ví dụ, và mỗi tập được cân bằng về mặt cảm xúc. Ma trận đặc trưng huấn luyện thu được X có kích thước 25.000×10.000, nhưng chỉ có 1,3% các phần tử nhị phân là khác 0. Chúng ta gọi ma trận như vậy là ma trận thưa, bởi vì hầu hết các giá trị đều giống nhau.

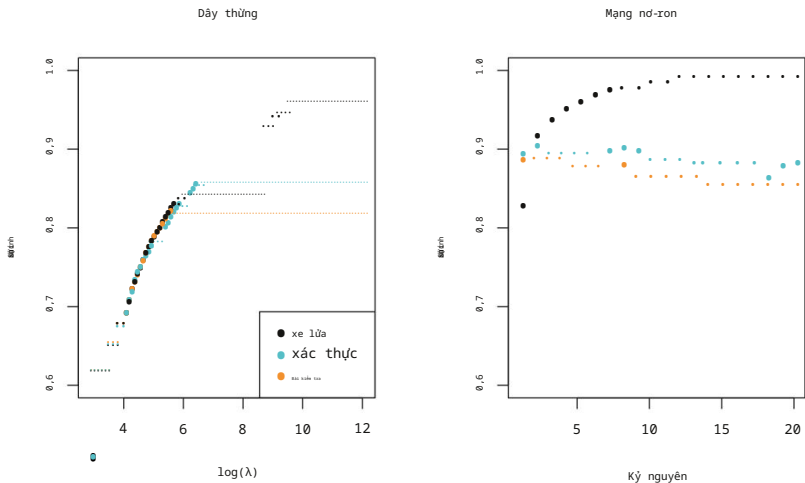
(bằng không trong trường hợp này); nó có thể được lưu trữ hiệu quả ở định dạng ma trận thưa. Có nhiều cách để tính đến độ dài của tài liệu; ở đây chúng ta chỉ chấm điểm một từ là có trong tài liệu hay không, nhưng ví dụ, người ta cũng có thể ghi lại tần suất tương đối của các từ. Chúng ta tách một tập dữ liệu xác thực có kích thước 2.000 từ 25.000 quan sát huấn luyện (để tính chính mô hình), và huấn luyện hai chuỗi mô hình:

14

định dạng ma trận thưa

- Mô hình hồi quy logistic lasso sử dụng gói glmnet ;
- Một mạng nơ-ron hai lớp với hai lớp ẩn, mỗi lớp có 16 Đơn vị ReLU.

14Thay vì lưu trữ toàn bộ ma trận, ta có thể lưu trữ vị trí và giá trị của các phần tử khác 0. Trong trường hợp này, vì tất cả các phần tử khác 0 đều bằng 1, nên chỉ lưu trữ vị trí của chúng.



HÌNH 10.11. Độ chính xác của thuật toán lasso và mạng nơ-ron hai lớp ẩn trên dữ liệu IMDB. Đối với lasso, trục x hiển thị $\log(\lambda)$, trong khi đối với mạng nơ-ron mạng này hiển thị các kỷ nguyên (số lần thuật toán khớp dữ liệu đi qua). (tập huấn luyện). Cả hai đều có xu hướng quá khớp và đạt được kết quả xấp xỉ

Độ chính xác của bài kiểm tra như nhau.

Cả hai phương pháp đều tạo ra một chuỗi các giải pháp. Chuỗi lasso được lập chỉ mục bởi tham số điều chỉnh λ . Chuỗi mạng nơ-ron được lập chỉ mục bởi số lần lặp giảm độ dốc được sử dụng trong quá trình khớp dữ liệu. được đo bằng số chu kỳ huấn luyện hoặc số lần duyệt qua tập dữ liệu huấn luyện (Mục 10.7). Lưu ý rằng độ chính xác huấn luyện trong Hình 10.11 (các điểm màu đen) Tăng dần đều trong cả hai trường hợp. Chúng ta có thể sử dụng lỗi xác thực để Chọn một giải pháp tốt từ mỗi chuỗi (các điểm màu xanh lam trong biểu đồ), giải pháp nào Sau đó, kết quả sẽ được sử dụng để đưa ra dự đoán trên tập dữ liệu thử nghiệm. Lưu ý rằng mạng nơ-ron hai lớp tương đương với một mô hình logistic phi tuyến tính. mô hình hồi quy. Từ (10.12) và (10.13) chúng ta có thể thấy rằng

$$\frac{\Pr(Y = 1|X)}{\Pr(Y = 0|X)} = \frac{Z_1}{Z_0} \tag{10.15}$$
$$= (\beta_{10} + \beta_{00}) + \sum_{k=1}^K (\beta_{1k} + \beta_{0k}) A_k(x)$$

(Điều này cho thấy sự dư thừa trong hàm softmax; đối với K lớp chúng ta Thực ra chỉ cần ước tính K - 1 bộ hệ số. (Xem Mục 4.3.5 .) Hình 10.11 thể hiện độ chính xác (tỷ lệ đúng) thay vì lỗi phân loại (tỷ lệ sai), trong đó độ chính xác phổ biến hơn trong máy học. cộng đồng học tập. Cả hai mô hình đều đạt độ chính xác trên tập dữ liệu thử nghiệm khoảng 88%. Mô hình túi từ tóm tắt một tài liệu dựa trên các từ có trong đó. và bỏ qua bối cảnh của chúng. Có ít nhất hai cách phổ biến để hiểu vấn đề này. cần xem xét bối cảnh:

- Mô hình túi n-gram . Ví dụ, một túi 2-gram ghi lại

416 10. Học sâu

Sự xuất hiện đồng thời liên tiếp của mỗi cặp từ khác nhau. "Dài một cách hạnh phúc" có thể được xem là một cụm từ tích cực trong bài phê bình phim, trong khi "ngắn một cách hạnh phúc" lại mang ý nghĩa tiêu cực.

- Hãy xem tài liệu như một chuỗi các sự kiện, lưu ý đến tất cả các từ trong ngữ cảnh của những từ đứng trước và những từ đứng sau.

Trong phần tiếp theo, chúng ta sẽ khám phá các mô hình cho chuỗi dữ liệu, có ứng dụng trong dự báo thời tiết, nhận dạng giọng nói, dịch ngôn ngữ và dự đoán chuỗi thời gian, cùng một số ứng dụng khác. Chúng ta sẽ tiếp tục với ví dụ [IMDb](#) ở đó.

10.5 Mạng thần kinh hồi quy

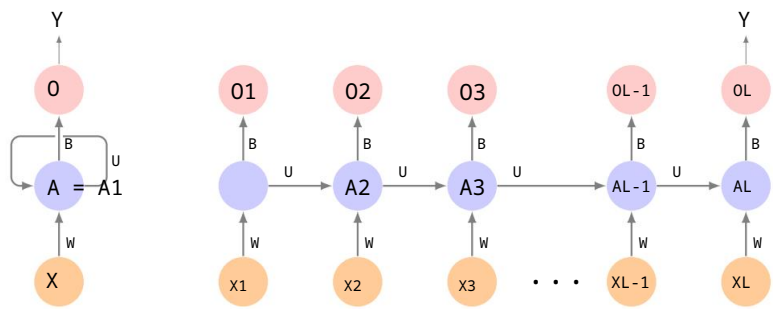
Nhiều nguồn dữ liệu có tính chất tuần tự và cần được xử lý đặc biệt khi xây dựng mô hình dự đoán. Ví dụ:

- Các tài liệu như bài đánh giá sách và phim, bài báo và bài đăng trên Twitter. Trình tự và vị trí tương đối của các từ trong tài liệu thể hiện nội dung, chủ đề và giọng điệu, và có thể được khai thác trong các nhiệm vụ như phân loại chủ đề, phân tích cảm xúc và dịch thuật.
- Chuỗi dữ liệu theo thời gian về nhiệt độ, lượng mưa, tốc độ gió, chất lượng không khí, v.v. Chúng ta có thể muốn dự báo thời tiết trước vài ngày hoặc dự báo khí hậu trước vài thập kỷ.
- Chuỗi thời gian tài chính, nơi chúng ta theo dõi các chỉ số thị trường, khối lượng giao dịch, giá cổ phiếu và trái phiếu, và tỷ giá hối đoái. Trong trường hợp này, việc dự đoán thường khó khăn, nhưng như chúng ta sẽ thấy, một số chỉ số nhất định có thể được dự đoán với độ chính xác hợp lý.
- Bản ghi âm giọng nói, bản ghi âm nhạc và các bản ghi âm khác. Chúng ta có thể muốn cung cấp bản ghi chép văn bản của một bài phát biểu, hoặc có thể là bản dịch ngôn ngữ. Chúng ta có thể muốn đánh giá chất lượng của một bản nhạc, hoặc gán một số thuộc tính nhất định.
- Chữ viết tay, chẳng hạn như ghi chú của bác sĩ, và các chữ số viết tay như mã bưu chính. Ở đây, chúng ta muốn chuyển đổi chữ viết tay thành văn bản kỹ thuật số, hay đọc các chữ số (nhận dạng ký tự quang học).

Trong mạng nơ-ron hồi quy (RNN), đối tượng đầu vào X là một chuỗi.

Hãy xem xét một tập hợp các tài liệu, chẳng hạn như bộ sưu tập các bài đánh giá phim [trên IMDb](#). Mỗi tài liệu có thể được biểu diễn dưới dạng một chuỗi L từ, tức là $X = \{X_1, X_2, \dots, X_L\}$, trong đó mỗi X đại diện cho một từ. Thứ tự của các từ và độ gần nhau của một số từ nhất định trong câu truyền tải ý nghĩa ngữ nghĩa. Mạng nơ-ron hồi quy (RNN) được thiết kế để thích ứng và tận dụng tính chất tuần tự của các đối tượng đầu vào như vậy, tương tự như cách mạng nơ-ron tích chập thích ứng với cấu trúc không gian của đầu vào hình ảnh. Đầu ra Y cũng có thể là một chuỗi (như trong dịch ngôn ngữ), nhưng thường là một giá trị vô hướng, chẳng hạn như nhãn cảm xúc nhị phân của một tài liệu đánh giá phim.

mạng thần
kinh hồi
quy



HÌNH 10.12. Sơ đồ mạng nơ-ron hồi quy đơn giản. Đầu vào là một chuỗi các vectơ $\{X\}_{L=1}^L$, và ở đây mục tiêu là một phản hồi duy nhất. Mạng xử lý chuỗi đầu vào X theo trình tự; mỗi X được đưa vào lớp ẩn, lớp này cũng nhận đầu vào là vectơ kích hoạt A_{t-1} từ phần tử trước đó trong chuỗi, và tạo ra vectơ kích hoạt hiện tại tại A_t . Cùng một tập hợp các trọng số W, U và B được sử dụng khi xử lý từng phần tử của chuỗi. Lớp đầu ra tạo ra một chuỗi các dự đoán O từ vectơ kích hoạt hiện tại tại A_t , nhưng thông thường chỉ có dự đoán cuối cùng trong số này, O_L , là có liên quan. Bên trái đầu bằng là một biểu diễn ngắn gọn của mạng, được mở rộng thành một phiên bản rõ ràng hơn ở bên phải.

Hình 10.12 minh họa cấu trúc của một mạng RNN rất cơ bản với chuỗi đầu vào $X = \{X_1, X_2, \dots, X_L\}$, đầu ra đơn giản Y và chuỗi lớp ẩn $\{A\}_{L=1}^L = \{A_1, A_2, \dots, A_L\}$. Mỗi X là một vectơ; trong ví dụ tài liệu, X có thể đại diện cho mã hóa one-hot cho từ thứ dựa trên từ điển ngôn ngữ của ngữ liệu (xem bảng trên cùng trong Hình 10.13 để biết ví dụ đơn giản). Khi chuỗi được xử lý từng vectơ X một, mạng sẽ cập nhật các kích hoạt A trong lớp ẩn, lấy đầu vào là vectơ X và vectơ kích hoạt A_{t-1} từ bước trước đó trong chuỗi. Mỗi A được đưa vào lớp đầu ra và tạo ra dự đoán O cho Y . O_L , dự đoán cuối cùng trong số này, là dự đoán quan trọng nhất.

Cụ thể, giả sử mỗi vectơ X của chuỗi đầu vào có p thành phần $X_T = (X_1, X_2, \dots, X_p)$, và lớp ẩn bao gồm K đơn vị $A_T = (A_1, A_2, \dots, A_K)$. Như trong Hình 10.4, chúng ta biểu diễn tập hợp $K \times (p+1)$ trọng số chung w_{kj} cho lớp đầu vào bằng ma trận W , và tương tự U là ma trận $K \times K$ của các trọng số u_{ks} cho các lớp ẩn-ẩn, và B là vectơ $K+1$ trọng số β_k cho lớp đầu ra. Khi đó

$$A_k = g \left(\sum_{j=1}^p w_{kj} X_j + \sum_{s=1}^K u_{ks} A_s \right) + \beta_k, \quad (10.16)$$

và đầu ra O được tính như sau:

$$O = \beta_0 + \sum_{k=1}^K \beta_k A_k \quad (10.17)$$

ví dụ, đối với phản hồi định lượng, hoặc với hàm kích hoạt sigmoid bổ sung cho phản hồi nhị phân. Ở đây $g(\cdot)$ là một hàm kích hoạt như ReLU. Lưu ý rằng các trọng số W, U và B giống nhau được sử dụng như chúng ta

418 10. Học sâu

Xử lý từng phần tử trong chuỗi, tức là chúng không phải là hàm của . Đây là một hình thức chia sẻ trọng số được sử dụng bởi RNN, và tương tự như việc sử dụng trọng số bộ lọc trong mạng nơ-ron tích chập (Mục 10.3.1). Khi chúng ta tiến hành từ đầu đến cuối, các kích hoạt A tích lũy lịch sử những gì đã được thấy trước đó, để ngữ cảnh đã học có thể được sử dụng để dự đoán.

Đối với các bài toán hồi quy, hàm mất mát cho một quan sát (X, Y) là

$$(Y - OL)^2, \tag{10.18}$$

chỉ tham chiếu đến đầu ra cuối cùng $OL = \beta_0 + \sum_{k=1}^K \beta_k AL_k$. Do đó, OL_1 không được sử dụng. Khi chúng ta điều chỉnh mô hình, mỗi phần tử X của chuỗi đầu vào X đóng góp vào OL thông qua chuỗi (10.16), và do đó gián tiếp đóng góp vào việc học các tham số chung W, U và B thông qua hàm mất mát (10.18). Với n cặp chuỗi đầu vào/phản hồi (x_i, y_i) , các tham số được tìm thấy bằng cách tối thiểu hóa tổng bình phương

$$\sum_{i=1}^N (y_i - o_i)^2 = \sum_{i=1}^N \left(y_i - \beta_0 - \sum_{k=1}^K \beta_k \sum_{j=1}^P w_{kj} x_{ij} + \sum_{s=1}^K u_{ks} a_{is} + b \right)^2. \tag{10.19}$$

Ở đây chúng ta sử dụng chữ cái thường cho các giá trị y_i quan sát được và các chuỗi vectơ $x_i = \{x_{i1}, x_{i2}, \dots, x_{iL}\}$,¹⁵ cũng như các kích hoạt được suy ra.

Vì các đầu ra trung gian O không được sử dụng, người ta có thể tự hỏi tại sao chúng lại tồn tại. Trước hết, chúng được tạo ra miễn phí, vì chúng sử dụng cùng trọng số đầu ra B cần thiết để tạo ra OL, và cung cấp dự đoán tiến hóa cho đầu ra. Hơn nữa, đối với một số nhiệm vụ học tập, phản hồi cũng là một chuỗi, và do đó chuỗi đầu ra $\{O_1, O_2, \dots, O_L\}$ là cần thiết một cách rõ ràng.

Khi được sử dụng ở mức độ mạnh nhất, mạng nơ-ron hồi quy có thể khá phức tạp. Chúng tôi minh họa việc sử dụng chúng trong hai ứng dụng đơn giản. Trong ứng dụng đầu tiên, chúng tôi tiếp tục với phân tích cảm xúc trên IMDb của phần trước, trong đó chúng tôi xử lý các từ trong các bài đánh giá theo trình tự. Trong ứng dụng thứ hai, chúng tôi minh họa việc sử dụng chúng trong bài toán dự báo chuỗi thời gian tài chính.

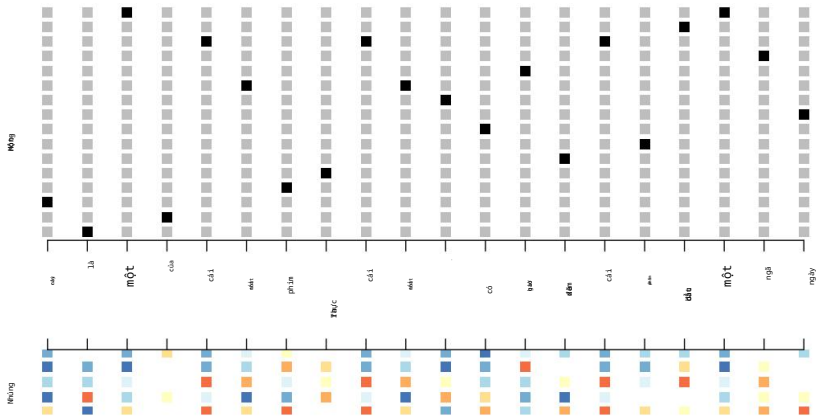
10.5.1 Mô hình tuần tự cho phân loại tài liệu

Ở đây, chúng ta quay lại với nhiệm vụ phân loại dựa trên các đánh giá của IMDb. Phương pháp chúng ta đã sử dụng trong Phần 10.4 là mô hình túi từ (bag-of-words). Ở đây, kế hoạch là thay vào đó sử dụng trình tự các từ xuất hiện trong tài liệu để dự đoán nhãn cho toàn bộ tài liệu.

Tuy nhiên, chúng ta gặp phải vấn đề về chiều dữ liệu: mỗi từ trong tài liệu của chúng ta được biểu diễn bằng một vectơ mã hóa one-hot (biến giả) với 10.000 phần tử (một phần tử cho mỗi từ trong từ điển)! Một phương pháp đang trở nên phổ biến là biểu diễn mỗi từ trong không gian nhúng có chiều thấp hơn nhiều. Điều này có nghĩa là thay vì biểu diễn mỗi từ bằng một vectơ nhị phân với 9.999 số 0 và một số 1 ở một vị trí nào đó, chúng ta sẽ biểu diễn nó bằng một tập hợp gồm m số thực, trong đó thường không có số nào bằng 0. Ở đây, m là chiều của không gian nhúng, và có thể nằm trong khoảng vài trăm, hoặc thậm chí ít hơn.

Điều này có nghĩa là (trong trường hợp của chúng ta) chúng ta cần một ma trận E có kích thước $m \times 10.000$.

¹⁵Đây là một dãy các vectơ; mỗi phần tử x_i là một vectơ p.



HÌNH 10.13. Mô tả một chuỗi 20 từ đại diện cho một tài liệu duy nhất: được mã hóa one-hot bằng cách sử dụng từ điển gồm 16 từ (hình trên) và được nhúng trong không gian m chiều với m = 5 (hình dưới).

trong đó mỗi cột được đánh số bởi một trong 10.000 từ trong từ điển của chúng ta, và các giá trị trong cột đó cho biết tọa độ m của từ đó trong không gian nhúng.

Hình 10.13 minh họa ý tưởng (với từ điển gồm 16 từ thay vì 10.000 từ, và m = 5). E đến từ đâu? Nếu chúng ta có một kho dữ liệu lớn gồm các tài liệu được gắn nhãn, chúng ta có thể cho mạng nơ-ron học E như một phần của quá trình tối ưu hóa. Trong trường hợp này, E được gọi là lớp nhúng, và một E chuyên biệt được học cho nhiệm vụ cụ thể. Nếu không, chúng ta có thể chèn một ma trận E được tính toán trước vào lớp nhúng, một quá trình được gọi là đồng bộ trọng số. Hai lớp nhúng được huấn luyện trước, word2vec và GloVe, được sử dụng rộng rãi. ¹⁶ Chúng được xây dựng từ một kho dữ liệu rất lớn gồm các tài liệu bằng một biến thể của phân tích thành phần chính (Mục 12.2). Ý tưởng là vị trí của các từ trong không gian nhúng bảo toàn ý nghĩa ngữ nghĩa; ví dụ: các từ đồng nghĩa nên xuất hiện gần nhau.

lớp nhúng

đồng lạnh
trọng lượng
word2vec
Găng tay

Cho đến nay, mọi thứ đều ổn. Mỗi tài liệu hiện được biểu diễn dưới dạng một chuỗi gồm m vectơ, mỗi vectơ biểu diễn trình tự các từ. Bước tiếp theo là giới hạn mỗi tài liệu chỉ còn L từ cuối cùng. Các tài liệu ngắn hơn L sẽ được thêm các số 0 vào đầu. Vì vậy, bây giờ mỗi tài liệu được biểu diễn bằng một chuỗi gồm L vectơ $X = \{X_1, X_2, \dots, X_L\}$, và mỗi X trong chuỗi có m thành phần.

Bây giờ chúng ta sử dụng cấu trúc RNN trong Hình 10.12. Tập dữ liệu huấn luyện bao gồm n chuỗi riêng biệt (tài liệu) có độ dài L, mỗi chuỗi được xử lý tuần tự từ trái sang phải. Trong quá trình này, một chuỗi song song các vectơ kích hoạt ẩn $A, = 1, \dots, L$ được tạo ra như trong (10.16) cho mỗi tài liệu. A được đưa vào lớp đầu ra để tạo ra dự đoán đang phát triển O. Chúng ta sử dụng giá trị cuối cùng O_L để dự đoán phản hồi: cảm xúc của bài đánh giá.

16 word2vec được mô tả trong Mikolov, Chen, Corrado, và Dean (2013), có sẵn tại <https://code.google.com/archive/p/word2vec>. GloVe được mô tả trong Pennington, Socher và Manning (2014), có sẵn tại <https://nlp.stanford.edu/projects/glove>.

420 10. Học sâu

Đây là một RNN đơn giản và có tương đối ít tham số. Nếu có K đơn vị ẩn, ma trận trọng số chung W có $K \times (m + 1)$ tham số, ma trận U có $K \times K$ tham số và B có $2(K + 1)$ cho hồi quy logistic hai lớp như trong (10.15). Chúng được sử dụng lặp đi lặp lại khi chúng ta xử lý chuỗi $X = \{X_t\}$ từ trái sang phải, giống như cách chúng ta sử dụng một bộ lọc tích chập duy nhất để xử lý từng mảng trong một hình ảnh (Mục 10.3.1). Nếu lớp nhúng E được học, điều đó sẽ thêm $m \times D$ tham số ($D = 10.000$ ở đây), và đây là chi phí lớn nhất.

Chúng tôi đã huấn luyện mô hình RNN như mô tả trong Hình 10.12 và phần văn bản kèm theo để phù hợp với dữ liệu IMDB. Mô hình có ma trận nhúng E với $m = 32$ (được học trong quá trình huấn luyện chứ không phải được tính toán trước), tiếp theo là một lớp hồi quy đơn với $K = 32$ đơn vị ẩn. Mô hình được huấn luyện với chuẩn hóa dropout trên 25.000 bài đánh giá trong tập huấn luyện được chỉ định và đạt được độ chính xác đáng thất vọng là 76% trên dữ liệu thử nghiệm IMDB. Một mạng sử dụng ma trận nhúng E được huấn luyện trước của GloVe cho hiệu suất tốt hơn một chút.

Để dễ trình bày, chúng tôi đã giới thiệu một mạng RNN rất đơn giản. Các phiên bản phức tạp hơn sử dụng bộ nhớ dài hạn và ngắn hạn (LSTM). Hai luồng kích hoạt lớp ẩn được duy trì, sao cho khi kích hoạt A được tính toán, nó nhận đầu vào từ các đơn vị ẩn cả ở xa hơn trong quá khứ và gần hơn trong quá khứ – được gọi là mạng RNN LSTM. Với các chuỗi dài, điều này khắc phục được vấn đề các tín hiệu ban đầu bị mờ đi trước khi chúng được truyền qua chuỗi đến vectơ kích hoạt cuối cùng AL.

Mạng nơ-ron hồi quy LSTM

Khi chúng tôi hiệu chỉnh lại mô hình bằng cách sử dụng kiến trúc LSTM cho lớp ẩn, hiệu suất đã được cải thiện lên 87% trên tập dữ liệu thử nghiệm IMDB. Con số này tương đương với mức 88% đạt được bởi mô hình bag-of-words trong Phần 10.4.

Chúng tôi sẽ cung cấp chi tiết về cách lắp đặt các mô hình này trong Mục 10.9.6.

Mặc dù có thêm sự phức tạp của LSTM, mạng RNN của chúng ta vẫn còn ở mức độ "cơ bản". Chúng ta có thể đạt được kết quả tốt hơn một chút bằng cách thay đổi kích thước mô hình, thay đổi tham số điều chỉnh và thêm các lớp ẩn. Tuy nhiên, mô hình LSTM mất nhiều thời gian để huấn luyện, điều này khiến việc khám phá nhiều kiến trúc và tối ưu hóa tham số trở nên tốn công.

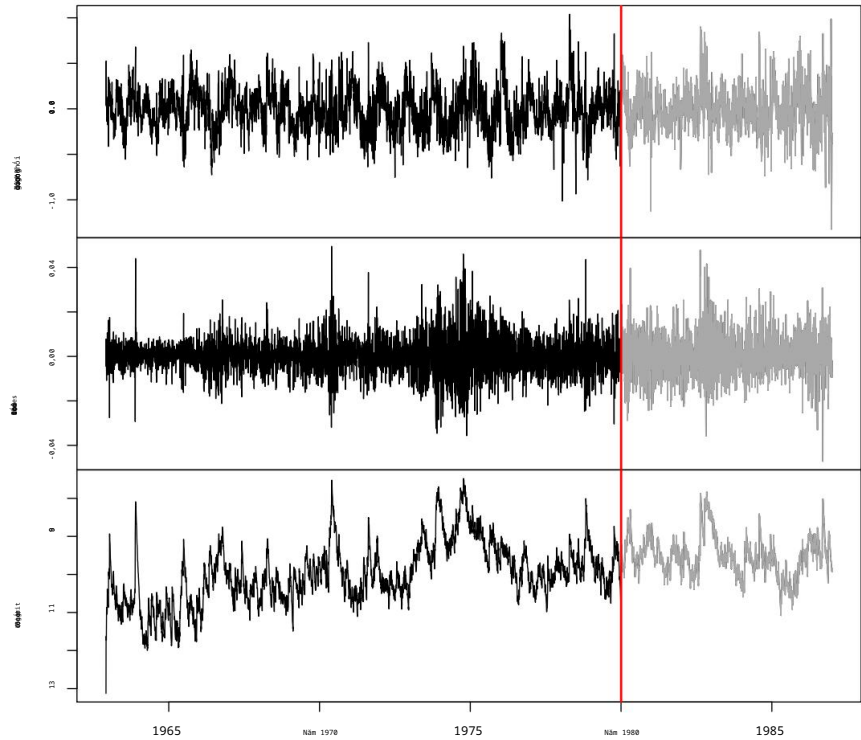
Mạng nơ-ron hồi quy (RNN) cung cấp một khuôn khổ phong phú để mô hình hóa các chuỗi dữ liệu, và chúng vẫn đang tiếp tục phát triển. Đã có nhiều tiến bộ trong việc phát triển RNN – về kiến trúc, tăng cường dữ liệu và thuật toán học. Tại thời điểm viết bài này (đầu năm 2020), các cấu hình RNN hàng đầu báo cáo độ chính xác trên 95% trên dữ liệu IMDB. Chi tiết cụ thể nằm ngoài phạm vi của cuốn sách này.

10.5.2 Dự báo chuỗi thời gian

Hình 10.14 thể hiện số liệu thống kê giao dịch lịch sử từ Sở Giao dịch Chứng khoán New York. Hình ảnh hiển thị ba chuỗi thời gian hàng ngày bao gồm giai đoạn từ ngày 3 tháng 12 năm 1962 đến ngày 31 tháng 12 năm 1986:18

¹⁷Bảng xếp hạng IMDB có thể được tìm thấy tại <https://paperswithcode.com/sota/sentiment-analysis-on-imdb>.

¹⁸Những dữ liệu này được tập hợp bởi LeBaron và Weigend (1998) Giao dịch IEEE về Mạng thần kinh, 9(1): 213-220.



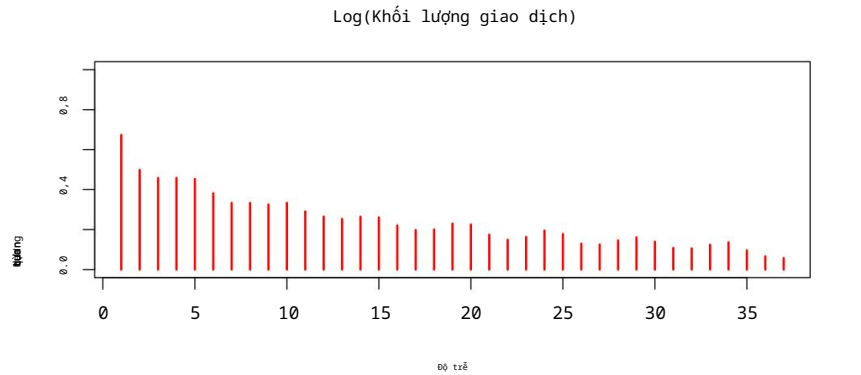
HÌNH 10.14. Thống kê giao dịch lịch sử từ Sở Giao dịch Chứng khoán New York. Giá trị hàng ngày của khối lượng giao dịch logarit chuẩn hóa, lợi nhuận của chỉ số DJIA và biến động logarit. Các số liệu được hiển thị trong khoảng thời gian 24 năm từ năm 1962 đến năm 1986. Chúng tôi muốn dự đoán khối lượng giao dịch vào bất kỳ ngày nào, dựa trên lịch sử của tất cả các ngày trước đó. Bên trái của thanh màu đỏ (tháng Giêng) (2, 1980) là dữ liệu huấn luyện, và bên phải là dữ liệu kiểm tra.

- **Khối lượng giao dịch logarit.** Đây là tỷ lệ phần trăm của tổng số cổ phiếu đang lưu hành mà được giao dịch trong ngày đó, so với mức trung bình động 100 ngày của các giao dịch trong quá khứ. Doanh thu, trên thang logarit.
- **Lợi nhuận của Dow Jones.** Đây là sự khác biệt giữa logarit cơ số 1 và logarit tự nhiên của chỉ số Dow Jones. Chỉ số Jones Industrial Index trong các ngày giao dịch liên tiếp.
- **Độ biến động logarit.** Chỉ số này dựa trên giá trị tuyệt đối của giá hàng ngày. sự chuyển động.

Dự đoán giá cổ phiếu là một vấn đề nổi tiếng khó khăn, nhưng hóa ra là Việc dự đoán khối lượng giao dịch dựa trên lịch sử giao dịch gần đây dễ quản lý hơn. (và rất hữu ích cho việc lập kế hoạch chiến lược giao dịch).

Một quan sát ở đây bao gồm các phép đo (vt, rt, zt) vào ngày t, trong Trong trường hợp này, các giá trị dành cho `log_volume`, `DJ_return` và `log_volatility` là... Tổng cộng có $T = 6.051$ bộ ba như vậy, mỗi bộ ba được biểu diễn dưới dạng chuỗi thời gian. Trong Hình 10.14, một đặc điểm gây ấn tượng ngay lập tức là các quan sát hàng ngày không độc lập với nhau. Chuỗi dữ liệu thể hiện... Tự tương quan – trong trường hợp này, các giá trị gần nhau về thời gian có xu hướng tương tự nhau. Điều này phân biệt chuỗi thời gian với các tập dữ liệu khác mà chúng ta có gặp phải, trong đó các quan sát có thể được giả định là độc lập với

tự do ng-
tương quan



HÌNH 10.15. Hàm tự tương quan cho `log_volume`. Ta thấy rằng Các giá trị lân cận có mối tương quan khá mạnh, với hệ số tương quan trên 0,2 ở khoảng cách xa. Cách nhau 20 ngày.

với nhau. Để rõ hơn, hãy xem xét các cặp quan sát ($v_t, v_{t+\tau}$), cách nhau một khoảng thời gian trễ là ngày. Nếu ta lấy tất cả các cặp như vậy trong chuỗi v_t và tính hệ số tương quan của chúng, ta sẽ có hệ số tự tương quan ở độ trễ τ . Hình 10.15 cho thấy Hàm tự tương quan cho tất cả các độ trễ lên đến 37, và chúng ta thấy sự khác biệt đáng kể. tương quan.

Một đặc điểm thú vị khác của bài toán dự báo này là...
Biến phản hồi $v_t - \log_volume$ – cũng là một biến dự đoán! Cụ thể, chúng ta sẽ sử dụng các giá trị `log_volume` trong quá khứ để dự đoán các giá trị trong tương lai.

Bộ dự báo RNN

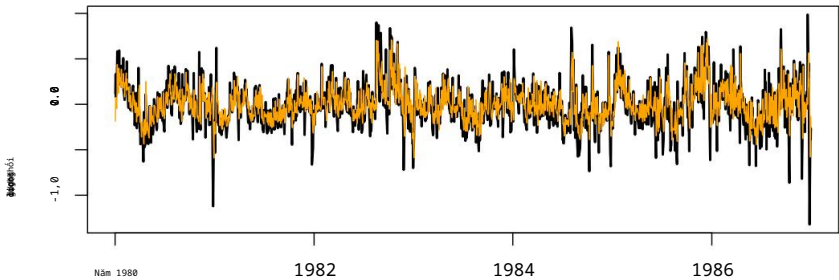
Chúng tôi muốn dự đoán giá trị v_t từ các giá trị trong quá khứ $v_1, v_2, \dots$, và cũng muốn Sử dụng các giá trị trong quá khứ của các chuỗi khác $r_1, r_2, \dots$ và $z_1, z_2, \dots$. Mặc dù dữ liệu tổng hợp của chúng tôi là một chuỗi khá dài với 6.051 giao dịch. Trong vài ngày tới, cấu trúc của bài toán khác với ví dụ phân loại tài liệu trước đó.

- Chúng ta chỉ có một chuỗi dữ liệu, chứ không phải 25.000 chuỗi.
- Chúng tôi có cả một loạt các mục tiêu v_t , và các đầu vào bao gồm quá khứ giá trị của chuỗi này.

Chúng ta có thể biểu diễn vấn đề này như thế nào dựa trên cấu trúc được hiển thị trong Hình 10.12? Ý tưởng là trích xuất nhiều chuỗi nhỏ gồm các chuỗi đầu vào $X = \{X_1, X_2, \dots, X_L\}$ với độ dài L được xác định trước (được gọi là độ trễ trong ngữ cảnh này), và mục tiêu Y tương ứng. Chúng có dạng

$$X_1 = \begin{matrix} v_t & L \\ r_t & L \\ z_t & L \end{matrix}, X_2 = \begin{matrix} v_{t+1} & L+1 \\ r_{t+1} & L+1 \\ z_{t+1} & L+1 \end{matrix}, \dots, X_L = \begin{matrix} v_{t+L} & 1 \\ r_{t+L} & 1 \\ z_{t+L} & 1 \end{matrix} \text{ và } Y = v_t. \tag{10.20}$$

Như vậy, ở đây mục tiêu Y là giá trị của `log_volume` v_t tại một thời điểm t duy nhất. và chuỗi đầu vào X là chuỗi các vectơ 3 chiều $\{X\}_L$, mỗi vectơ bao gồm trong số ba phép đo `log_volume`, `DJ_return` và `log_volatility` từ ngày $t - L$, $t - L + 1$, cho đến $t - 1$. Mỗi giá trị của t tạo thành một (X, Y) riêng biệt. cặp, với t chạy từ $L + 1$ đến T . Đối với dữ liệu `NYSE`, chúng ta sẽ sử dụng dữ liệu quá khứ.



HÌNH 10.16. Dự báo RNN về log_volume trên dữ liệu thử nghiệm NYSE. Các đường màu đen thể hiện khối lượng thực tế, và các đường màu cam chồng lên nhau thể hiện các dự báo. Chuỗi dự báo chiếm 42% phương sai của log_volume.

Năm ngày giao dịch để dự đoán khối lượng giao dịch của ngày tiếp theo. Do đó, chúng ta sử dụng $L = 5$. Vì $T = 6.051$, chúng ta có thể tạo ra 6.046 cặp (X, Y) như vậy. Rõ ràng L là một tham số cần được lựa chọn cẩn thận, có thể sử dụng dữ liệu kiểm chứng.

Chúng tôi đã huấn luyện mô hình này với $K = 12$ đơn vị ẩn bằng cách sử dụng 4.281 chuỗi huấn luyện được tạo ra từ dữ liệu trước ngày 2 tháng 1 năm 1980 (xem Hình 10.14), và sau đó sử dụng nó để dự báo 1.770 giá trị của log_volume sau ngày này. Chúng tôi đạt được hệ số $R^2 = 0,42$ trên dữ liệu thử nghiệm. Chi tiết được trình bày trong Mục 10.9.6. Để đơn giản hóa, việc sử dụng giá trị log_volume của ngày hôm qua làm dự đoán cho ngày hôm nay cho kết quả $R^2 = 0,18$. Hình 10.16 thể hiện kết quả dự báo. Chúng tôi đã vẽ các giá trị quan sát được của log_volume hàng ngày trong giai đoạn thử nghiệm 1980-1986 bằng màu đen, và chồng lên chuỗi dự đoán bằng màu cam. Sự tương ứng có vẻ khá tốt.

Để dự báo giá trị của log_volume trong giai đoạn thử nghiệm, chúng ta phải sử dụng chính dữ liệu thử nghiệm để tạo ra chuỗi đầu vào X . Điều này có vẻ như là gian lận, nhưng thực tế không phải vậy; chúng ta luôn sử dụng dữ liệu quá khứ để dự đoán tương lai.

Tự hồi quy

Mạng nơ-ron hồi quy (RNN) mà chúng ta vừa xây dựng có nhiều điểm chung với mô hình tự hồi quy (AR) tuyến tính truyền thống, mà chúng ta sẽ trình bày ngay bây giờ để so sánh. Trước tiên, chúng ta xem xét riêng chuỗi phản hồi vt và xây dựng vectơ phản hồi y cùng ma trận M các biến dự đoán cho hồi quy bình phương tối thiểu như sau:

</

424 10. Học sâu

của cùng một chuỗi. Việc hồi quy y theo M tương đương với việc điều chỉnh mô hình.

$$\hat{v}_t = \beta^0 + \beta^1 v_{t-1} + \beta^2 v_{t-2} + \dots + \beta^L v_{t-L}, \tag{10.22}$$

và được gọi là mô hình tự hồi quy bậc L, hoặc đơn giản là AR(L). Đối với dữ liệu *NYSE*, chúng ta có thể bao gồm các phiên bản trễ của *DJ_return* và *log_volatility*, *rt* và *zt*, trong ma trận dự đoán M, dẫn đến 3L + 1 cột. Mô hình AR với L = 5 đạt được R² kiểm định là 0,41, thấp hơn một chút so với 0,42 đạt được bởi RNN.

Dĩ nhiên, mô hình RNN và AR rất giống nhau. Cả hai đều sử dụng cùng một phản hồi Y và chuỗi đầu vào X có độ dài L = 5 và chiều p = 3 trong trường hợp này. RNN xử lý chuỗi này từ trái sang phải với cùng trọng số W (cho lớp đầu vào), trong khi mô hình AR chỉ đơn giản là coi tất cả L phần tử của chuỗi như một vectơ gồm L × p phần tử dự đoán – một quá trình được gọi là làm phẳng trong tài liệu về mạng nơ-ron.

làm phẳng

Tất nhiên RNN cũng bao gồm các kích hoạt lớp ẩn A truyền thông tin dọc theo chuỗi và giới thiệu thêm tính phi tuyến tính. Từ (10.19) với K = 12 đơn vị ẩn, chúng ta thấy rằng RNN có 13 + 12 × (1 + 3 + 12) = 205 tham số, so với 16 tham số cho mô hình AR(5).

Một phần mở rộng rõ ràng của mô hình AR là sử dụng tập hợp các yếu tố dự báo trễ làm vectơ đầu vào cho một mạng nơ-ron truyền thẳng thông thường (10.1), và do đó bổ sung thêm tính linh hoạt. Điều này đạt được R² thử nghiệm = 0,42, tốt hơn một chút so với AR tuyến tính và tương đương với RNN.

Tất cả các mô hình đều có thể được cải thiện bằng cách thêm biến *day_of_week* tương ứng với ngày t của biến mục tiêu *vt* (có thể được học từ ngày tháng trên lịch được cung cấp cùng với dữ liệu); khối lượng giao dịch thường cao hơn vào thứ Hai và thứ Sáu. Vì có năm ngày giao dịch, nên mã hóa one-hot này được chuyển đổi thành năm biến nhị phân. Hiệu suất của mô hình AR được cải thiện lên R² = 0,46, tương tự như mô hình RNN, và mô hình AR phi tuyến tính được cải thiện lên R² = 0,47.

Chúng tôi đã sử dụng phiên bản đơn giản nhất của mạng RNN trong các ví dụ ở đây. Các thí nghiệm bổ sung với phần mở rộng LSTM của RNN đã mang lại những cải tiến nhỏ, thường lên đến 1% về R² trong các ví dụ này.

Chúng tôi sẽ trình bày chi tiết cách chúng tôi lắp ráp cả ba mô hình trong Mục 10.9.6.

10.5.3 Tóm tắt về RNN

Chúng ta đã minh họa RNN thông qua hai trường hợp sử dụng đơn giản, và mới chỉ chạm đến bề nổi của nó.

Có rất nhiều biến thể và cải tiến của mạng RNN đơn giản mà chúng ta đã sử dụng để mô hình hóa chuỗi. Một phương pháp mà chúng ta chưa thảo luận sử dụng mạng nơ-ron tích chập một chiều, coi chuỗi các vectơ (ví dụ như từ, được biểu diễn trong không gian nhúng) như một hình ảnh. Bộ lọc tích chập trượt dọc theo chuỗi theo cách một chiều, với tiềm năng học các cụm từ cụ thể hoặc các chuỗi con ngắn có liên quan đến nhiệm vụ học tập.

Người ta cũng có thể có thêm các lớp ẩn trong mạng RNN. Ví dụ, với hai lớp ẩn, chuỗi A được coi là chuỗi đầu vào cho lớp ẩn tiếp theo một cách rõ ràng.

Mạng nơ-ron hồi quy (RNN) mà chúng tôi sử dụng đã quét tài liệu từ đầu đến cuối; thay thế-
Mạng RNN hai chiều bản địa quét các chuỗi theo cả hai hướng. hai chiều

Trong dịch thuật ngôn ngữ, mục tiêu cũng là một chuỗi từ, trong một
Ngôn ngữ khác với ngôn ngữ của chuỗi đầu vào. Cả chuỗi đầu vào và chuỗi mục tiêu đều được biểu diễn
bằng một cấu trúc tương tự như

Hình 10.12, và chúng chia sẻ các đơn vị ẩn. Trong phương pháp học Seq2Seq này, các đơn vị ẩn được Seq2Seq
cho là nắm bắt ý nghĩa ngữ nghĩa.

của các câu. Một số bước đột phá lớn trong mô hình ngôn ngữ và
Việc dịch thuật này là kết quả của những cải tiến tương đối gần đây trong các mạng nơ-ron hồi quy (RNN) như vậy.

Các thuật toán được sử dụng để huấn luyện mạng nơ-ron hồi quy (RNN) có thể phức tạp và tốn kém về mặt tính toán.
May mắn thay, phần mềm tốt phần nào giúp người dùng tránh được những phức tạp này, và làm cho việc
xác định và điều chỉnh các mô hình trở nên tương đối dễ dàng. Nhiều

Các mô hình mà chúng ta sử dụng hàng ngày (như Google Translate) đều dựa trên kiến trúc hiện đại
được phát triển bởi các nhóm kỹ sư tay nghề cao, và
đã được đào tạo bằng cách sử dụng nguồn tài nguyên tính toán và dữ liệu khổng lồ.

10.6 Khi nào nên sử dụng học sâu

Hiệu suất của học sâu trong chương này khá ấn tượng. Nó đã giải quyết thành công bài toán phân loại
chữ số, và mạng CNN sâu thực sự đã chứng tỏ được khả năng vượt trội.

Nó đã cách mạng hóa việc phân loại hình ảnh. Chúng ta thấy hàng ngày những báo cáo về những câu
chuyện thành công mới của học sâu. Nhiều trong số đó liên quan đến phân loại hình ảnh.

các nhiệm vụ như chẩn đoán tự động ảnh chụp nhũ ảnh hoặc ảnh X-quang kỹ thuật số,
Các bản quét nhãn khoa, chú thích ảnh chụp cộng hưởng từ (MRI), v.v. Tương tự như vậy.

Có rất nhiều thành công của mạng nơ-ron hồi quy (RNN) trong việc dịch thuật giọng nói và ngôn ngữ.
dự báo và mô hình hóa tài liệu. Câu hỏi đặt ra là: liệu chúng ta có nên loại bỏ tất cả các công cụ
cũ và sử dụng học sâu cho mọi thứ hay không?

Có vấn đề gì với dữ liệu? Để giải quyết câu hỏi này, chúng ta xem xét lại bộ dữ liệu Hitters của mình.
Trích từ Chương 6.

Đây là một bài toán hồi quy, trong đó mục tiêu là dự đoán mức lương của...
Một cầu thủ bóng chày vào năm 1987 sử dụng số liệu thống kê thành tích của anh ấy từ năm 1986. Sau đó
Sau khi loại bỏ những người chơi có câu trả lời bị thiếu, chúng ta còn lại 263 người chơi và
19 biến số. Chúng tôi chia ngẫu nhiên dữ liệu thành tập huấn luyện gồm 176 người chơi.
(hai phần ba), và một bộ dữ liệu thử nghiệm gồm 87 người chơi (một phần ba). Chúng tôi đã sử dụng ba phương pháp.
để áp dụng mô hình hồi quy cho dữ liệu này.

- Mô hình tuyến tính được sử dụng để khớp dữ liệu huấn luyện và đưa ra dự đoán.
trên dữ liệu thử nghiệm. Mô hình có 20 tham số.
- Mô hình tuyến tính tương tự đã được điều chỉnh bằng phương pháp lasso. Việc điều chỉnh
Tham số được chọn bằng phương pháp kiểm chứng chéo 10 lần trên tập dữ liệu huấn luyện.
dữ liệu. Nó đã chọn một mô hình với 12 biến có hệ số khác 0.
- Một mạng nơ-ron với một lớp ẩn gồm 64 đơn vị ReLU.
Mô hình này được điều chỉnh phù hợp với dữ liệu. Mô hình này có 1.345 tham số.20

20Mô hình được hiệu chỉnh bằng phương pháp giảm độ dốc ngẫu nhiên với kích thước lô là 32 cho 1.000
số epoch và tỷ lệ dropout 10%. Hiệu suất lỗi kiểm thử ổn định lại và
Bắt đầu tăng dần sau 1.000 chu kỳ. Các chi tiết về việc khớp dữ liệu này được thảo luận trong Phần 10.7.

Mô hình	# Tham số	Sai số Trung bình Tuyệt đối	Sai số Tập kiểm tra R2
Hồi quy tuyến tính	20	254,7	0,56
Dây thừng	12	252,3	0,51
Mạng thần kinh	1345	257,4	0,54

BẢNG 10.2. Kết quả dự đoán trên dữ liệu thử nghiệm **Hitters** cho các mô hình tuyến tính phù hợp bằng phương pháp bình phương tối thiểu thông thường và phương pháp Lasso, so sánh với mạng nơ-ron được huấn luyện bằng phương pháp ngẫu nhiên. Phương pháp giảm độ dốc với điều chỉnh dropout.

	Hệ số Sai số chuẩn	Thống kê t	giá trị p
Chặn	-226,67	86,26	-2,63 0,0103
Lượt tay cấp	3,06	1,02	3.00 0.0036
Đi bộ	0,181	2,04	0,09 0,9294
CRuns	0,859	0,12	7,09 < 0,0001
PutOuts	0,465	0,13	3,60 0,0005

BẢNG 10.3. Ước tính hệ số bình phương nhỏ nhất liên quan đến hồi quy **Lượng** trên bốn biến được chọn bằng phương pháp lasso trên tập dữ liệu thử nghiệm **Hitters**. Điều này Mô hình đạt hiệu suất tốt nhất trên dữ liệu thử nghiệm, với sai số tuyệt đối trung bình là của 224,8. Kết quả được báo cáo ở đây thu được từ phép hồi quy trên bài kiểm tra dữ liệu này không được sử dụng để hiệu chỉnh mô hình lasso.

Bảng 10.2 so sánh các kết quả. Chúng ta thấy hiệu suất tương tự nhau đối với cả ba phương pháp. các mô hình. Chúng tôi báo cáo sai số tuyệt đối trung bình trên dữ liệu thử nghiệm, cũng như Hệ số R2 của phép thử cho mỗi phương pháp đều đáng tin cậy (xem Bài tập 5). Chúng tôi đã dành khá nhiều thời gian để mày mò với các thông số cấu hình của mạng nơ-ron đã đạt được những kết quả này. Có thể là nếu chúng ta Dành nhiều thời gian hơn, và điều chỉnh hình thức cũng như số lượng điều chỉnh cho phù hợp. rằng chúng ta có thể đạt được hoặc thậm chí vượt trội hơn so với hồi quy tuyến tính và cái thông lọng. Nhưng chúng tôi đã dễ dàng thu được các mô hình tuyến tính hoạt động tốt. Các mô hình tuyến tính dễ trình bày và hiểu hơn nhiều so với các mô hình mạng nơ-ron. mạng lưới, về bản chất là một hộp đen. Công cụ lasso đã chọn 12 trong số 19. các biến số trong việc đưa ra dự đoán. Vì vậy, trong những trường hợp như thế này, chúng ta sẽ tốt hơn nhiều. Tuân theo nguyên tắc dao cạo Occam : khi đối mặt với nhiều phương pháp cho hiệu quả tương đương nhau, hãy chọn phương pháp đơn giản nhất.

Occam's
dao cạo

Sau khi tìm hiểu thêm một chút với mô hình lasso, chúng tôi đã xác định được một kết quả thậm chí còn tốt hơn. mô hình đơn giản hơn với bốn biến. Sau đó, chúng tôi hiệu chỉnh lại mô hình tuyến tính với các biến này. bốn biến số được đưa vào dữ liệu huấn luyện (còn gọi là phương pháp lasso nơi lỏng), và đạt được Với sai số tuyệt đối trung bình của bài kiểm tra là 224,8, người chiến thắng chung cuộc! Thật hấp dẫn khi... Trình bày bảng tóm tắt từ kết quả phù hợp này, để chúng ta có thể thấy các hệ số và giá trị p; tuy nhiên, vì mô hình được chọn trên dữ liệu huấn luyện, nên... sẽ dẫn đến sai lệch chọn lọc. Thay vào đó, chúng tôi hiệu chỉnh lại mô hình trên dữ liệu thử nghiệm, điều này không được sử dụng trong quá trình lựa chọn. Bảng 10.3 hiển thị kết quả.

Chúng ta có một số công cụ rất mạnh mẽ trong tay, bao gồm cả mạng lưới thần kinh. Mạng nơ-ron, rừng ngẫu nhiên và thuật toán boosting, máy vectơ hỗ trợ và mô hình cộng tính tổng quát, chỉ là một vài ví dụ. Và sau đó chúng ta có các mô hình tuyến tính. và các biến thể đơn giản của chúng. Khi đối mặt với các vấn đề mô hình hóa dữ liệu và dự đoán mới, người ta thường có xu hướng lựa chọn những phương pháp mới nhất. Chúng thường cho kết quả vô cùng ấn tượng, đặc biệt là khi các tập dữ liệu chúng rất lớn và có thể hỗ trợ việc khớp các mô hình phi tuyến đa chiều. các mô hình. Tuy nhiên, nếu chúng ta có thể tạo ra các mô hình bằng các công cụ đơn giản hơn thì

Nếu hiệu năng tương đương, chúng thường dễ lắp đặt và hiểu hơn, và có khả năng ít bị lỗi hơn so với các phương pháp phức tạp hơn. Bất cứ khi nào có thể, nên thử cả các mô hình đơn giản hơn, rồi đưa ra lựa chọn dựa trên sự cân bằng giữa hiệu năng và độ phức tạp.

Thông thường, chúng ta kỳ vọng học sâu sẽ là một lựa chọn hấp dẫn khi kích thước mẫu của tập dữ liệu huấn luyện cực kỳ lớn và khi khả năng giải thích của mô hình không phải là ưu tiên hàng đầu.

10.7 Xây dựng mạng nơ-ron



Việc xây dựng mô hình mạng nơ-ron khá phức tạp, và chúng tôi sẽ trình bày tổng quan ngắn gọn ở đây. Các ý tưởng này có thể áp dụng cho các mạng phức tạp hơn nhiều. Những độc giả nào thấy nội dung này khó hiểu có thể bỏ qua phần này. May mắn thay, như chúng ta thấy trong phần thực hành ở cuối chương này, đã có phần mềm tốt để xây dựng mô hình mạng nơ-ron một cách tương đối tự động, mà không cần lo lắng về các chi tiết kỹ thuật của quy trình xây dựng mô hình.

Chúng ta bắt đầu với mạng lưới đơn giản được mô tả trong Hình 10.1 ở Phần 10.1. Trong mô hình (10.1), các tham số là $\beta = (\beta_0, \beta_1, \dots, \beta_K)$, cũng như mỗi $w_k = (w_{k0}, w_{k1}, \dots, w_{kp})$, $k = 1, \dots, K$. Với các quan sát (x_i, y_i) , $i = 1, \dots, n$, chúng ta có thể điều chỉnh mô hình bằng cách giải bài toán bình phương tối thiểu phi tuyến tính.

$$\min_{\beta, w_k} \sum_{i=1}^n (y_i - f(x_i))^2, \quad (10.23)$$

Ở đây

$$f(x_i) = \beta_0 + \sum_{k=1}^K \sum_{j=1}^p w_{kj} x_{ij}. \quad (10.24)$$

Mục tiêu trong (10.23) trông khá đơn giản, nhưng do sự sắp xếp lồng nhau của các tham số và tính đối xứng của các đơn vị ẩn, việc tối thiểu hóa nó không hề đơn giản. Bài toán là phi lồi đối với các tham số, và do đó có nhiều nghiệm. Ví dụ, Hình 10.17 cho thấy một hàm phi lồi đơn giản của một biến θ ; có hai nghiệm: một là cực tiểu địa phương và một là cực tiểu toàn cục. Hơn nữa, (10.1) là mạng nơ-ron đơn giản nhất; trong chương này, chúng tôi đã trình bày những mạng phức tạp hơn nhiều, trong đó các vấn đề này được kết hợp toàn cục. Để khắc phục một số vấn đề này và để bảo vệ khỏi hiện tượng quá khớp, hai chiến lược chung được sử dụng khi huấn luyện mạng nơ-ron.

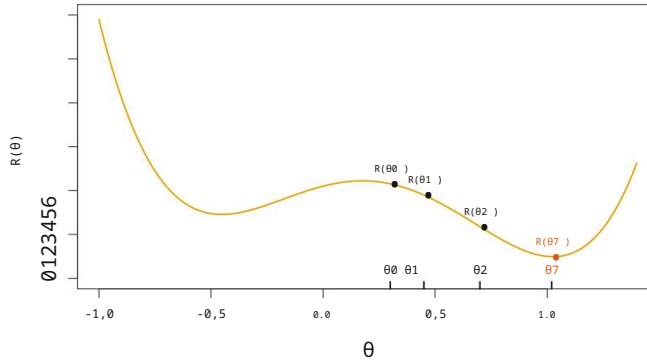
địa phương
tối thiểu
tối thiểu

- Học chậm: mô hình được huấn luyện theo cách lặp đi lặp lại khá chậm, sử dụng phương pháp giảm độ dốc. Quá trình huấn luyện sẽ dừng lại khi phát hiện hiện tượng quá khớp độ dốc.

- Điều chỉnh tham số: các hình phạt được áp đặt lên các tham số, thường là lasso hoặc sống núi như đã thảo luận trong Phần 6.2.

Giả sử chúng ta biểu diễn tất cả các tham số trong một vectơ dài θ . Khi đó, chúng ta có thể viết lại mục tiêu trong (10.23) như sau:

$$R(\theta) = \frac{1}{2} \sum_{i=1}^n (y_i - f_{\theta}(x_i))^2, \quad (10.25)$$



HÌNH 10.17. Minh họa thuật toán giảm độ dốc cho θ một chiều. Hàm mục tiêu $R(\theta)$ không lồi và có hai điểm cực tiểu, một tại $\theta = 0,46$ (cục bộ), điểm còn lại tại $\theta = 1,02$ (toàn cục). Bắt đầu từ một giá trị θ_0 nào đó (thường được chọn ngẫu nhiên), mỗi bước trong θ đi chuyển xuống dốc – ngược chiều độ dốc – cho đến khi không thể xuống thấp hơn nữa. Ở đây, thuật toán giảm độ dốc đạt đến điểm cực tiểu toàn cục sau 7 bước.

Trong đó, chúng ta làm rõ sự phụ thuộc của f vào các tham số. Ý tưởng của phương pháp giảm độ dốc rất đơn giản.

1. Bắt đầu với một giá trị dự đoán θ_0 cho tất cả các tham số trong θ , và đặt $t = 0$.
2. Lặp lại cho đến khi mục tiêu (10.25) không giảm nữa:

- (a) Tìm một vectơ δ phản ánh một sự thay đổi nhỏ trong θ , sao cho $\theta_{t+1} = \theta_t + \delta$ làm giảm mục tiêu; tức là sao cho $R(\theta_{t+1}) < R(\theta_t)$. (b) Đặt $t = t + 1$.

Ta có thể hình dung (Hình 10.17) việc đứng trên một địa hình đồi núi, và mục tiêu là xuống đến chân đồi thông qua một loạt các bậc thang. Miễn là mỗi bậc thang đều đi xuống dốc, cuối cùng ta sẽ xuống đến chân đồi. Trong trường hợp này, chúng ta may mắn, bởi vì với giá trị dự đoán ban đầu θ_0 , chúng ta đạt được điểm cực tiểu toàn cục. Nói chung, chúng ta có thể hy vọng đạt được một điểm cực tiểu cục bộ (tốt).

10.7.1 Lan truyền ngược

Làm thế nào để chúng ta tìm ra hướng di chuyển θ sao cho giảm mục tiêu $R(\theta)$ trong (10.25)? Độ dốc của $R(\theta)$, được đánh giá tại một giá trị hiện tại $\theta = \theta_m$, độ dốc là vectơ đạo hàm riêng tại điểm đó:

$$R(\theta_m) = \theta \frac{R(\theta)}{\theta = \theta_m} \quad (10.26)$$

Chỉ số dưới $\theta = \theta_m$ có nghĩa là sau khi tính toán vectơ đạo hàm, ta đánh giá nó tại giá trị dự đoán hiện tại, θ_m . Điều này cho biết hướng trong không gian θ mà $R(\theta)$ tăng nhanh nhất. Ý tưởng của phương pháp giảm độ dốc là di chuyển θ một chút theo hướng ngược lại (vì ta muốn đi xuống dốc):

$$\theta_{m+1} = \theta_m - \rho R(\theta_m) \quad (10.27)$$

Với giá trị tốc độ học ρ đủ nhỏ, bước này sẽ làm giảm mục tiêu $R(\theta)$; tức là $R(\theta_{m+1}) \leq R(\theta_m)$. Nếu vectơ gradient bằng không, thì Chúng ta có thể đã đạt được mức tối thiểu của mục tiêu.

Phép tính (10.26) phức tạp đến mức nào? Hóa ra nó khá phức tạp. Đơn giản ở đây, và vẫn đơn giản ngay cả đối với các mạng phức tạp hơn nhiều. do quy tắc chuỗi của phép vi phân.

Vì $R(\theta) = \sum_{i=1}^N Ri(\theta) = \frac{1}{2} \sum_{i=1}^N (y_i - f(\theta(x_i)))^2$ là tổng, đạo hàm của nó cũng là tổng của n quan sát, vì vậy chúng ta sẽ chỉ xem xét một trong số đó.

các điều khoản,

$$Ri(\theta) = \frac{1}{2} (y_i - \sum_{k=1}^K \beta_k \sum_{j=1}^P w_{kj} x_{ij})^2 \quad (10.28)$$

Để đơn giản hóa các biểu thức tiếp theo, ta viết $z_{ik} = \sum_{j=1}^P w_{kj} x_{ij}$. Đầu tiên, ta lấy đạo hàm theo β_k :

$$\begin{aligned} \frac{\partial Ri(\theta)}{\partial \beta_k} &= \frac{\partial Ri(\theta)}{\partial f(\theta(x_i))} \cdot \frac{\partial f(\theta(x_i))}{\partial \beta_k} \\ &= (y_i - f(\theta(x_i))) \cdot g(z_{ik}). \end{aligned} \quad (10.29)$$

Và bây giờ chúng ta lấy đạo hàm theo w_{kj} :

$$\begin{aligned} \frac{\partial Ri(\theta)}{\partial w_{kj}} &= \frac{\partial Ri(\theta)}{\partial f(\theta(x_i))} \cdot \frac{\partial f(\theta(x_i))}{\partial g(z_{ik})} \cdot \frac{\partial g(z_{ik})}{\partial w_{kj}} \cdot \frac{\partial z_{ik}}{\partial w_{kj}} \\ &= (y_i - f(\theta(x_i))) \cdot \beta_k \cdot g'(z_{ik}) \cdot x_{ij}. \end{aligned} \quad (10.30)$$

Lưu ý rằng cả hai biểu thức này đều chứa phần dư $y_i - f(\theta(x_i))$. Trong (10.29) chúng ta thấy rằng một phần của phần dư đó được phân bổ cho mỗi các đơn vị ẩn theo giá trị của $g(z_{ik})$. Sau đó, trong (10.30) chúng ta thấy một sự gán tương tự cho đầu vào j thông qua đơn vị ẩn k . Vì vậy, hành động vi phân gán một phần của phần dư cho mỗi tham số thông qua Quy tắc chuỗi – một quy trình được gọi là lan truyền ngược trong tài liệu về mạng nơ-ron. Mặc dù các phép tính này khá đơn giản, nhưng nó đòi hỏi sự cẩn thận. Sổ sách kế toán dùng để theo dõi tất cả các khoản mục.

tốc độ học tập

quy tắc chuỗi

lan truyền ngược

lô nhỏ

xuống

10.7.2 Chuẩn hóa và thuật toán giảm độ dốc ngẫu nhiên

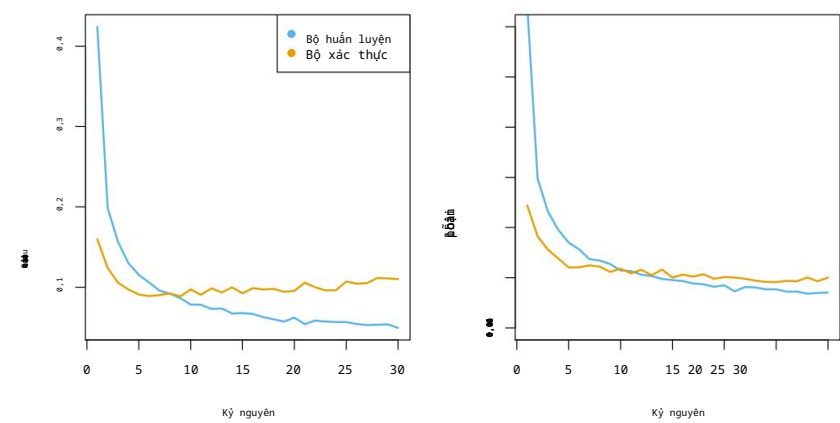
Thuật toán gradient descent thường cần nhiều bước để đạt đến điểm cực tiểu cục bộ. Trong thuật toán này, Trên thực tế, có một số phương pháp để đẩy nhanh quá trình này. Ngoài ra, khi n lớn, thay vì cộng (10.29)-(10.30) trên tất cả n quan sát, chúng ta có thể lấy mẫu một phần nhỏ hoặc một lô nhỏ trong số chúng mỗi khi tính toán một bước gradient. Quá trình này được gọi là gradient ngẫu nhiên

phương pháp giảm dần (SGD) và là phương pháp tiên tiến nhất để học mạng nơ-ron sâu. Ngẫu nhiên

May mắn thay, hiện có phần mềm rất tốt để thiết lập mô hình học sâu gradient.

các phương thức và cách thức điều chỉnh chúng cho phù hợp với dữ liệu, do đó hầu hết các chi tiết kỹ thuật đều được ẩn đi. Từ người dùng.

Bây giờ chúng ta sẽ xem xét mạng đa lớp (Hình 10.4) được sử dụng trong kỹ thuật số. vấn đề nhận dạng. Mạng có hơn 235.000 trọng số, tức là khoảng Số lượng ví dụ huấn luyện gấp bốn lần. Chuẩn hóa là điều cần thiết ở đây.



HÌNH 10.18. Sự tiến triển của lỗi huấn luyện và lỗi xác thực đối với mạng nơ-ron MNIST. Mạng lưới được mô tả trong Hình 10.4, là một hàm của số chu kỳ huấn luyện. Mục tiêu đề cập đến logarit xác suất (10.14).

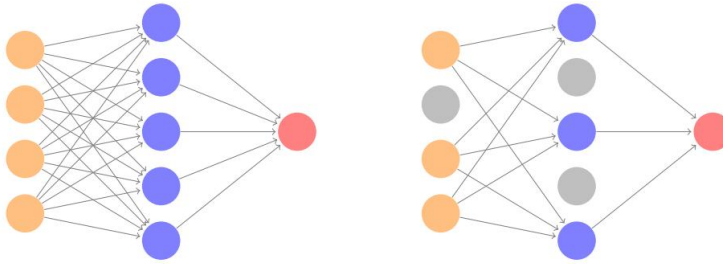
để tránh hiện tượng quá khớp. Hàng đầu tiên trong Bảng 10.1 sử dụng phương pháp điều chỉnh Ridge trên trọng số. Điều này đạt được bằng cách tăng cường hàm mục tiêu (10.14) kèm theo điều khoản phạt:

$$R(\theta; \lambda) = \sum_{i=1}^N \log(f_{\theta}(x_i)) + \lambda \sum_{j=1}^M \theta_j^2. \tag{10.31}$$

Tham số λ thường được thiết lập sẵn ở một giá trị nhỏ, hoặc được tìm thấy bằng cách sử dụng... Phương pháp tập dữ liệu xác thực của Mục 5.3.1. Chúng ta cũng có thể sử dụng các giá trị khác nhau của λ đại diện cho các nhóm trọng lượng từ các lớp khác nhau; trong trường hợp này là W_1 và W_2 . đã bị phạt, trong khi số lượng trọng số B tương đối ít của lớp đầu ra thì... Không bị phạt gì cả. Phương pháp điều chỉnh Lasso cũng phổ biến như một phương pháp bổ sung. một hình thức chuẩn hóa, hoặc như một giải pháp thay thế cho đường gờ.

Hình 10.18 hiển thị một số chỉ số thay đổi trong quá trình huấn luyện. Mạng lưới trên dữ liệu MNIST. Hóa ra SGD tự nhiên thực thi điều đó. hình thức riêng của phương pháp điều chỉnh xấp xỉ bậc hai.²¹ Ở đây, minibatch Kích thước là 128 quan sát cho mỗi lần cập nhật độ dốc. Thuật ngữ "epochs" được dùng để ghi nhận "epochs" trong Hình 10.18 đếm số lần tương đương với... Toàn bộ tập dữ liệu huấn luyện đã được xử lý. Đối với mạng này, 20% trong số 60.000 Dữ liệu quan sát quá trình huấn luyện được sử dụng làm tập dữ liệu xác thực nhằm mục đích xác định Khi nào nên dừng huấn luyện. Trên thực tế, 48.000 quan sát đã được sử dụng cho quá trình huấn luyện, và do đó có $48.000/128 \approx 375$ lần cập nhật độ dốc theo minibatch. mỗi epoch. Chúng ta thấy rằng giá trị của mục tiêu xác thực thực sự bắt đầu để tăng thêm 30 epoch, do đó việc dừng sớm cũng có thể được sử dụng như một hình thức điều chỉnh bổ sung.

21. Tính chất này và các tính chất khác của SGD đối với học sâu là chủ đề của nhiều nghiên cứu. trong các tài liệu về học máy tại thời điểm viết bài này.



HÌNH 10.19. Học Dropout. Bên trái: mạng nơ-ron kết nối đầy đủ. Bên phải: mạng nơ-ron với dropout ở lớp đầu vào và lớp ẩn. Các nút màu xám được chọn.

một cách ngẫu nhiên và bị bỏ qua trong một trường hợp huấn luyện.

10.7.3 Học tập bỏ học

Hàng thứ hai trong Bảng 10.1 được ghi nhận là dropout. Đây là một hình thức điều chỉnh tương đối mới và hiệu quả, tương tự ở một số khía cạnh với điều chỉnh ridge. Lấy cảm hứng từ rừng ngẫu nhiên (Mục 8.2), ý tưởng là loại bỏ ngẫu nhiên một phần φ các đơn vị trong một lớp khi huấn luyện mô hình.

Hình 10.19 minh họa điều này. Việc này được thực hiện riêng biệt mỗi khi có buổi huấn luyện.

Quá trình quan sát được xử lý. Các đơn vị còn sống sót sẽ thay thế cho những đơn vị bị mất tích.

và trọng lượng của chúng được nhân lên theo hệ số $1/(1 - \varphi)$ để bù lại.

Điều này ngăn chặn các nút trở nên quá chuyên biệt và có thể được xem như là

một hình thức điều chỉnh. Trên thực tế, dropout được thực hiện bằng cách đặt ngẫu nhiên các

giá trị kích hoạt cho các đơn vị "bị loại bỏ" về 0, trong khi vẫn giữ nguyên các giá trị khác.

Kiến trúc vẫn còn nguyên vẹn.

bỏ học

10.7.4 Điều chỉnh mạng

Mạng lưới trong Hình 10.4 được coi là tương đối đơn giản;

Tuy nhiên, điều đó đòi hỏi một số lựa chọn, và tất cả đều có ảnh hưởng đến...

hiệu suất:

- Số lượng lớp ẩn và số lượng đơn vị trên mỗi lớp.
Quan điểm hiện đại cho rằng số lượng đơn vị trên mỗi lớp ẩn có thể có thể lớn, và hiện tượng quá khớp có thể được kiểm soát thông qua nhiều hình thức khác nhau của sự điều chỉnh.
- Các tham số điều chỉnh chuẩn hóa. Chúng bao gồm tỷ lệ bỏ qua φ và độ mạnh λ của phương pháp điều chỉnh lasso và ridge, và thường là được thiết lập riêng biệt ở mỗi lớp.
- Chi tiết về thuật toán giảm độ dốc ngẫu nhiên. Bao gồm kích thước lô (batch size), số lượng kỷ nguyên, và nếu có sử dụng, chi tiết về việc tăng cường dữ liệu (Mục 10.3.4).

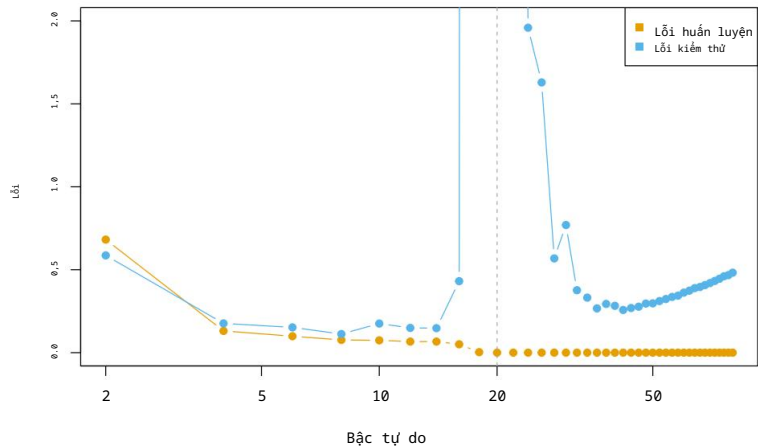
Những lựa chọn như vậy có thể tạo ra sự khác biệt. Trong quá trình chuẩn bị ví dụ MNIST này,

chúng tôi đã đạt được tỷ lệ sai sót phân loại khá tốt là 1,8% sau một số thử nghiệm.

và lỗi. Việc tinh chỉnh và huấn luyện một mạng lưới tương tự có thể được thực hiện dễ dàng hơn.

Sai số chỉ 1% trên các dữ liệu này, nhưng quá trình tinh chỉnh có thể tốn nhiều thời gian và công sức.

Nếu thực hiện không cẩn thận sẽ dẫn đến hiện tượng quá khớp (overfitting).



HÌNH 10.20. Hiện tượng giảm kép, được minh họa bằng biểu đồ lỗi cho một Ví dụ về đường cong spline tự nhiên một chiều. Trục ngang biểu thị số lượng của các hàm cơ sở spline trên thang logarit. Sai số huấn luyện đạt bằng không khi... bậc tự do trùng với kích thước mẫu $n = 20$, “phép nội suy” ngưỡng”, và sau đó vẫn giữ nguyên bằng không. Sai số kiểm tra tăng lên đáng kể. ở ngưỡng này, nhưng sau đó lại giảm xuống một giá trị hợp lý trước khi cuối cùng đang tăng trở lại.

10.8 Nội suy và Giảm độ cao kép

Trong suốt cuốn sách này, chúng ta đã nhiều lần thảo luận về sự đánh đổi giữa độ lệch và phương sai, lần đầu tiên được trình bày trong Phần 2.2.2. Sự đánh đổi này cho thấy rằng thống kê Các phương pháp học có xu hướng hoạt động tốt nhất, xét về lỗi trên tập dữ liệu kiểm tra, đối với một mức độ phức tạp trung bình của mô hình. Cụ thể, nếu ta vẽ “độ linh hoạt” trên trục x và sai số trên trục y, thì nhìn chung ta sẽ thấy Sai số kiểm thử có dạng chữ U, trong khi sai số huấn luyện giảm dần một cách đơn điệu. Hai ví dụ “điển hình” về hành vi này có thể được thấy ở phía bên phải.

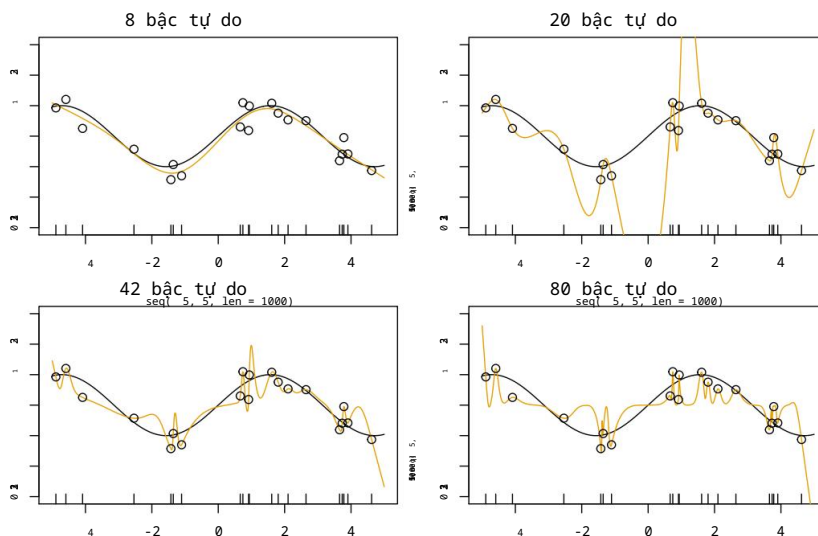
bảng của Hình 2.9 trên trang 29 và trong Hình 2.17 trên trang 39. Một hệ quả của sự đánh đổi giữa độ lệch và phương sai là nói chung không phải là một ý tưởng hay khi Nội suy dữ liệu huấn luyện – tức là để đạt được lỗi huấn luyện bằng không – vì điều đó thường dẫn đến lỗi kiểm thử rất cao. nội suy

Tuy nhiên, hóa ra trong một số trường hợp cụ thể thì điều đó hoàn toàn có thể xảy ra. một phương pháp học thống kê nội suy dữ liệu huấn luyện để thực hiện tốt – hoặc ít nhất, tốt hơn một mô hình đơn giản hơn một chút mà không Nội suy dữ liệu khá đầy đủ. Hiện tượng này được gọi là giảm kép. và được hiển thị trong Hình 10.20. “Phương pháp giảm kép” có tên gọi như vậy. Thực tế là lỗi kiểm tra có hình chữ U trước ngưỡng nội suy. đạt đến đỉnh, rồi lại hạ xuống (ít nhất là trong một thời gian) như một sự gia tăng ngày càng lớn. Mô hình linh hoạt phù hợp.

Chúng tôi sẽ mô tả thiết lập dẫn đến Hình 10.20. Chúng tôi đã mô phỏng $n = 20$ quan sát từ mô hình

$$Y = \sin(X) + \epsilon,$$

trong đó $X \sim U[-5, 5]$ (phân bố đều), và $\epsilon \sim N(0, \sigma^2)$ với $\sigma = 0.3$. Sau đó, chúng tôi sử dụng hàm spline tự nhiên để khớp dữ liệu, như mô tả trong Phần 7.4, với d



HÌNH 10.21. Các hàm được hiệu chỉnh $\hat{f}_d(X)$ (màu cam), hàm thực $f(X)$ (màu đen) và 20 điểm dữ liệu huấn luyện được quan sát. Một giá trị khác của d (bậc tự do) được sử dụng trong mỗi bảng. Đối với $d \geq 20$, tất cả các đường cong màu cam đều nội suy quá trình huấn luyện. điểm, và do đó lỗi huấn luyện bằng không.

bậc tự do.22 Nhớ lại từ Phần 7.4 rằng việc khớp một đường cong spline tự nhiên với d bậc tự do tương đương với việc sử dụng hồi quy bình phương tối thiểu của phân hồi lên một tập hợp các hàm cơ sở d . Bảng phía trên bên trái của Hình 10.21 hiển thị dữ liệu, hàm thực $f(X)$ và $\hat{f}_8(X)$, hàm được điều chỉnh Đường cong spline tự nhiên với $d = 8$ bậc tự do.

Tiếp theo, chúng ta sử dụng hàm spline tự nhiên với $d = 20$ bậc tự do. Vì $n = 20$, Điều này có nghĩa là $n = d$, và chúng ta có lỗi huấn luyện bằng không; nói cách khác, chúng ta đã nội suy dữ liệu huấn luyện! Chúng ta có thể thấy điều đó từ bảng phía trên bên phải của Hình 10.21 cho thấy $\hat{f}_{20}(X)$ có những biến động lớn, và do đó gây ra lỗi kiểm tra. Sẽ rất lớn.

Chúng tôi tiếp tục điều chỉnh các đường cong spline tự nhiên cho phù hợp với dữ liệu, với các giá trị tăng dần của d . Đối với $d > 20$, phương trình hồi quy bình phương nhỏ nhất của Y lên các hàm cơ sở d . Điều này không phải là duy nhất: có vô số ước lượng hệ số bình phương tối thiểu đạt được sai số bằng không. Để lựa chọn trong số đó, chúng ta chọn một trong số đó với tổng bình phương hệ số nhỏ nhất, $\sum_{j=1}^d \beta_j^2$. Điều này được gọi là Giải pháp chuẩn tối thiểu.

Hai bảng phía dưới của Hình 10.21 hiển thị chuẩn tối thiểu tự nhiên Đường cong spline khớp với $d = 42$ và $d = 80$ bậc tự do. Thật đáng kinh ngạc, $\hat{f}_{42}(X)$ ít hoang dã hơn đáng kể so với $\hat{f}_{20}(X)$, mặc dù nó sử dụng nhiều hơn bậc tự do. Và $\hat{f}_{80}(X)$ cũng không khác biệt nhiều. Tại sao lại như vậy? Về cơ bản, $\hat{f}_{20}(X)$ rất khó xác định vì chỉ có một cách duy nhất để nội suy $n = 20$ quan sát bằng cách sử dụng $d = 20$ hàm cơ sở, và cách duy nhất đó là... kết quả là một hàm được điều chỉnh khá cực đoan. Ngược lại, có một

22 Điều này ngụ ý việc lựa chọn d nút, ở đây được chọn tại d phân vị xác suất bằng nhau của dữ liệu huấn luyện. Khi $d > n$, các phân vị được tìm thấy bằng phương pháp nội suy.

434 10. Học sâu

có vô số cách nội suy $n = 20$ quan sát bằng cách sử dụng $d = 42$ hoặc $d = 80$ hàm cơ sở, và cách mượt nhất trong số đó – tức là giải pháp chuẩn tối thiểu – ít biến động hơn nhiều so với $\hat{f}_{20}(X)$!

Trong Hình 10.20, chúng ta hiển thị lỗi huấn luyện và lỗi kiểm tra liên quan đến $\hat{f}_d(X)$, cho một loạt các giá trị của bậc tự do d . Chúng ta thấy rằng lỗi huấn luyện giảm xuống bằng không khi $d = 20$ trở lên; tức là khi đạt đến ngưỡng nội suy. Ngược lại, lỗi kiểm tra cho thấy hình chữ U đối với $d \leq 20$, tăng lên rất lớn xung quanh $d = 20$, và sau đó cho thấy một vùng giảm thứ hai đối với $d > 20$. Đối với ví dụ này, tỷ lệ tín hiệu trên nhiễu – $\text{Var}(f(X))/\sigma^2$ – là 5,9, khá cao (các điểm dữ liệu gần với đường cong thực). Vì vậy, một ước tính nội suy dữ liệu và không đi quá xa giữa các điểm dữ liệu quan sát được có khả năng sẽ hoạt động tốt.

Trong Hình 10.20 và 10.21, chúng ta đã minh họa hiện tượng giảm kép trong một thiết lập một chiều đơn giản bằng cách sử dụng spline tự nhiên. Tuy nhiên, hóa ra hiện tượng tương tự cũng có thể xảy ra đối với học sâu. Về cơ bản, khi huấn luyện mạng nơ-ron với số lượng tham số khổng lồ, đôi khi chúng ta có thể đạt được kết quả tốt với lỗi huấn luyện bằng không. Điều này đặc biệt đúng trong các bài toán có tỷ lệ tín hiệu trên nhiễu cao, chẳng hạn như nhận dạng hình ảnh tự nhiên và dịch ngôn ngữ. Điều này là do các kỹ thuật được sử dụng để huấn luyện mạng nơ-ron, bao gồm cả thuật toán giảm độ dốc ngẫu nhiên, tự nhiên phù hợp với việc lựa chọn một mô hình nội suy "mượt mà" có hiệu suất tốt trên tập dữ liệu kiểm thử đối với các loại bài toán này.

Có một số điểm cần nhấn mạnh:

- Hiện tượng giảm kép không mâu thuẫn với sự đánh đổi giữa độ lệch và phương sai, như đã trình bày trong Phần 2.2.2. Thay vào đó, đường cong giảm kép được thấy ở phía bên phải của Hình 10.20 là hệ quả của việc trục x hiển thị số lượng hàm cơ sở spline được sử dụng, điều này không thể hiện đúng "tính linh hoạt" thực sự của các mô hình nội suy dữ liệu huấn luyện. Nói cách khác, trong ví dụ này, spline tự nhiên chuẩn tối thiểu với $d = 42$ có phương sai thấp hơn spline tự nhiên với $d = 20$.

- Hầu hết các phương pháp học thống kê được trình bày trong cuốn sách này không thể hiện hiện tượng suy giảm kép. Ví dụ, các phương pháp điều chỉnh thường không nội suy dữ liệu huấn luyện, do đó hiện tượng suy giảm kép không xảy ra. Đây không phải là nhược điểm của các phương pháp điều chỉnh: chúng có thể cho kết quả tuyệt vời mà không cần nội suy dữ liệu!

Cụ thể, trong các ví dụ ở đây, nếu chúng ta sử dụng hồi quy Ridge với mức phạt được chọn phù hợp thay vì phương pháp bình phương tối thiểu để khớp các hàm spline tự nhiên, thì chúng ta sẽ không thấy hiện tượng giảm kép, và trên thực tế sẽ thu được kết quả lỗi kiểm định tốt hơn.

- Trong Chương 9, chúng ta đã thấy rằng các bộ phân loại biên độ tối đa và SVM có lỗi huấn luyện bằng không vẫn thường đạt được lỗi kiểm tra rất tốt. Điều này một phần là do các phương pháp đó tìm kiếm các giải pháp có chuẩn tối thiểu mượt mà. Điều này tương tự như việc spline tự nhiên có chuẩn tối thiểu có thể cho kết quả tốt với lỗi huấn luyện bằng không.
- Hiện tượng giảm kép đã được cộng đồng học máy sử dụng để giải thích thực tiễn thành công của việc sử dụng quá mức...

mạng nơ-ron tham số hóa (nhiều lớp và nhiều đơn vị ẩn), sau đó điều chỉnh cho đến khi lỗi huấn luyện bằng không. Tuy nhiên, việc điều chỉnh cho đến khi lỗi bằng không không phải lúc nào cũng tối ưu, và liệu điều đó có nên hay không phụ thuộc vào tỷ lệ tín hiệu trên nhiễu. Ví dụ, chúng ta có thể sử dụng chuẩn hóa Ridge để tránh hiện tượng quá khớp mạng nơ-ron, như trong (10.31). Trong trường hợp này, với điều kiện chúng ta sử dụng lựa chọn thích hợp cho tham số điều chỉnh λ , chúng ta sẽ không bao giờ nội suy dữ liệu huấn luyện, và do đó sẽ không thấy hiện tượng giảm kép. Tuy nhiên, chúng ta có thể đạt được hiệu suất tập kiểm tra rất tốt, có thể tốt hơn nhiều so với trường hợp chúng ta nội suy dữ liệu huấn luyện. Việc dừng sớm trong quá trình giảm độ dốc ngẫu nhiên cũng có thể đóng vai trò là một hình thức chuẩn hóa giúp ngăn chặn việc nội suy dữ liệu huấn luyện, trong khi vẫn đạt được kết quả rất tốt trên dữ liệu kiểm tra.

Tóm lại: mặc dù hiện tượng suy giảm kép đôi khi có thể xảy ra trong mạng nơ-ron, nhưng thông thường chúng ta không muốn dựa vào hành vi này. Hơn nữa, điều quan trọng cần nhớ là sự đánh đổi giữa độ lệch và phương sai luôn đúng (mặc dù có thể lỗi kiểm tra như một hàm của tính linh hoạt có thể không thể hiện hình chữ U, tùy thuộc vào cách chúng ta đã tham số hóa khái niệm "tính linh hoạt" trên trục x).

10.9 Thực hành: Học sâu

Trong phần này, chúng ta sẽ trình bày cách thức để phù hợp với các ví dụ đã được thảo luận trong văn bản. Chúng ta sử dụng gói ``torch`` của `Python`, cùng với gói ``pytorch_lightning`` cung cấp các tiện ích để đơn giản hóa việc phù hợp và đánh giá mô hình. Mã này có thể chạy rất nhanh với một số bộ xử lý đặc biệt, chẳng hạn như chip M1 mới của Apple. Gói này được cấu trúc tốt, linh hoạt và sẽ dễ sử dụng đối với người dùng `Python`. Một nguồn tham khảo hữu ích là trang web ``pytorch.org/tutorials``. Phần lớn mã của chúng tôi được điều chỉnh từ đó, cũng như tài liệu `pytorch_lightning`.²³ Chúng tôi bắt đầu với một số lệnh nhập tiêu chuẩn mà chúng tôi đã thấy trước đây.

ngon đước
pytorch_
lightning

```
Trong [1]: import numpy as np, pandas as pd
from matplotlib.pyplot import subplots from
sklearn.linear_model import \
    (LinearRegression,
     LogisticRegression,
     Lasso)

from sklearn.preprocessing import StandardScaler from
sklearn.model_selection import KFold from
sklearn.pipeline import Pipeline from ISLP
import load_data from ISLP.models
import ModelSpec as MS from sklearn.model_selection
import \ (train_test_split, GridSearchCV)
```

23. Các URL chính xác tại thời điểm viết bài này là <https://pytorch.org/tutorials/beginner/basics/intro.html> và <https://pytorch-lightning.readthedocs.io/en/latest/>.

Nhập khẩu dành riêng cho Torch

Có một số mặt hàng nhập khẩu dành cho `đèn pin`. (Những mặt hàng này không được bao gồm trong...) (`ISLP`, vì vậy cần phải được cài đặt riêng.) Đầu tiên, chúng ta nhập thư viện chính. và các công cụ thiết yếu được sử dụng để xác định các mạng có cấu trúc tuần tự.

Trong [2]: nhập torch

```
from torch import nn
from torch.optim import RMSprop
from torch.utils.data import TensorDataset
```

Có một số gói hỗ trợ khác cho `torch`. Ví dụ, Gói `torchmetrics` có các tiện ích để tính toán nhiều chỉ số khác nhau nhằm đánh giá hiệu suất khi huấn luyện mô hình. Gói `torchinfo` cung cấp bản tóm tắt hữu ích về các lớp của mô hình. Chúng tôi sử dụng hàm `read_image()` khi tải ảnh thử nghiệm trong Mục 10.9.4.

`torchmetrics`
thông tin về đèn pin
`đọc_hình ảnh()`

Trong [3]: từ `torchmetrics` nhập (`MeanAbsoluteError`,
`R2Score`)

`from torchinfo import summary`
`from torchvision.io import read_image`

Gói `pytorch_lightning` là một giao diện ở cấp độ cao hơn một chút cho `đèn pin` giúp đơn giản hóa việc xác định thông số kỹ thuật và lắp đặt các mô hình bằng cách giảm thiểu... Lượng mã mẫu cần thiết (so với chỉ sử dụng `torch`).

Trong [4]: từ `pytorch_lightning` nhập `Trainer`
`from pytorch_lightning.loggers import CSVLogger`

Để tái tạo kết quả, chúng ta sử dụng `seed_everything()`. Chúng ta cũng sẽ... Hướng dẫn `torch` sử dụng các thuật toán xác định khi có thể.

hạt giống_
mọi thứ()

Trong [5]: từ `pytorch_lightning.utilities.seed` import `seed_everything`
`giao hạt mọi thứ (0, người lao động=True)`
`torch.use_deterministic_algorithms(True, warn_only=True)`

Chúng ta sẽ sử dụng một số tập dữ liệu được cung cấp kèm theo `torchvision` cho các ví dụ của mình: một mạng được huấn luyện trước để phân loại hình ảnh, cũng như một số phép biến đổi. Được sử dụng cho giai đoạn tiền xử lý.

tầm nhìn đèn pin

Trong [6]: từ `torchvision.datasets` nhập `MNIST`, `CIFAR100`
`from torchvision.models import (resnet50,`
`ResNet50_Weights)`

`from torchvision.transforms import (Resize,`
`Chuẩn hóa,`
`CenterCrop,`
`ToTensor)`

Chúng tôi đã cung cấp một vài tiện ích trong `ISLP` dành riêng cho phòng thí nghiệm này. `SimpleDataModule` và `SimpleModule` là các phiên bản đơn giản của các đối tượng được sử dụng. Trong `pytorch_lightning`, mô-đun cấp cao dùng để hiệu chỉnh các mô hình `torch` . Mặc dù có những ứng dụng nâng cao hơn như tính toán trên xử lý đồ họa. Các đơn vị xử lý đồ họa (GPU) và xử lý dữ liệu song song đều khả thi trong mô-đun này. Sẽ không tập trung nhiều vào những vấn đề này trong bài thực hành này. Bộ xử lý lỗi (`ErrorTracker`) sẽ xử lý chúng . tập hợp các mục tiêu và dự đoán trên mỗi lô nhỏ trong quá trình xác thực hoặc giai đoạn thử nghiệm, cho phép tính toán chỉ số trong toàn bộ quá trình xác thực. hoặc bộ dữ liệu thử nghiệm.

Trong [7]: từ ISLP.torch nhập (SimpleDataModule,
SimpleModule,
ErrorTracker,
rec_num_workers)

Ngoài ra, chúng tôi đã bổ sung một số hàm hỗ trợ để tải cơ sở dữ liệu IMDb, cũng như một hàm tra cứu ảnh xạ các số nguyên đến các khóa cụ thể trong cơ sở dữ liệu. Chúng tôi đã bao gồm một bản sao được sửa đổi đôi chút của dữ liệu IMDb đã được xử lý trước từ `keras`, một gói riêng biệt để huấn luyện các mô hình học sâu. Điều này giúp chúng ta tiết kiệm đáng kể thời gian xử lý trước và cho phép chúng ta tập trung vào việc xác định và huấn luyện các mô hình. `keras`

Trong [8]: từ ISLP.torch.imdb nhập (load_lookup, load_tensor,
load_sparse,
load_sequential)

Cuối cùng, chúng tôi giới thiệu một số thư viện tiện ích không liên quan trực tiếp đến `torch`. Hàm `glob()` từ mô-đun `glob` được sử dụng để tìm tất cả các tệp khớp với các ký tự đại diện, mà chúng ta sẽ sử dụng trong ví dụ áp dụng mô hình `ResNet50` cho một số hình ảnh của riêng chúng ta. `glob()` Mô-đun `json` sẽ được sử dụng để tải một tệp JSON để tra cứu các lớp nhằm xác định nhãn của các hình ảnh trong ví dụ `ResNet50`. `json`

Trong [9]: từ `glob` nhập `glob` nhập
`json`

10.9.1 Mạng một lớp trên dữ liệu người đánh bóng

Chúng ta bắt đầu bằng cách áp dụng các mô hình trong Phần 10.6 vào dữ liệu của đội `Hitters`.

Trong [10]: `Hitters = load_data('Hitters').dropna() n =
Hitters.shape[0]`

Chúng ta sẽ xây dựng hai mô hình tuyến tính (phương pháp bình phương tối thiểu và phương pháp Lasso) và so sánh hiệu suất của chúng với mạng nơ-ron. Để so sánh, chúng ta sẽ sử dụng sai số tuyệt đối trung bình trên tập dữ liệu kiểm chứng.

$$MAE(y, \hat{y}) = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|.$$

Chúng tôi thiết lập ma trận mô hình và phản hồi.

Trong [11]: `model = MS(Hitters.columns.drop('Salary'), intercept=False)
X = model.fit_transform(Hitters).to_numpy()
Y = Hitters['Salary'].to_numpy()`

Phương thức `'to_numpy()'` ở trên chuyển đổi các data frame hoặc series của `pandas` thành các mảng `numpy`. Chúng ta làm điều này vì sẽ cần sử dụng `sklearn` để huấn luyện mô hình lasso, và điều này yêu cầu chuyển đổi đó. Chúng ta cũng sử dụng phương pháp hồi quy tuyến tính từ `sklearn`, thay vì phương pháp trong Chương 3 từ `statsmodels`, để thuận tiện cho việc so sánh.

Giờ chúng ta chia dữ liệu thành tập kiểm tra và tập huấn luyện, cố định trạng thái ngẫu nhiên. Được `sklearn` sử dụng để thực hiện việc chia tách.

```
Trong [12]: (X_train,
           X_test,
           Y_train,
           Y_test) = train_test_split(X,
                                     Y, test_size=1/3,
                                     random_state=1)
```

Mô hình tuyến tính

Chúng tôi áp dụng mô hình tuyến tính và đánh giá trực tiếp sai số kiểm định.

```
Trong [13]: hit_lm = LinearRegression().fit(X_train, Y_train)
           Yhat_test = hit_lm.predict(X_test)
           np.abs(Yhat_test - Y_test).mean()
```

Ra[13]: 259.7153

Tiếp theo, chúng ta sẽ huấn luyện thuật toán lasso bằng thư viện `sklearn`. Chúng ta sử dụng sai số tuyệt đối trung bình (MAE) để chọn và đánh giá mô hình, thay vì sai số bình phương trung bình (MASE). Thuật toán giải chuyên dụng mà chúng ta đã sử dụng trong Mục 6.5.2 chỉ sử dụng MASE. Vì vậy, ở đây, với một chút công sức nữa, chúng ta sẽ tạo một lưới kiểm định chéo và thực hiện kiểm định chéo trực tiếp.

Chúng tôi mã hóa một quy trình gồm hai bước: đầu tiên, chúng tôi chuẩn hóa các đặc trưng bằng phép biến đổi `StandardScaler()`, sau đó huấn luyện lasso mà không cần chuẩn hóa thêm.

```
Trong [14]: scaler = StandardScaler(with_mean=True, with_std=True) lasso =
           Lasso(warm_start=True, max_iter=30000) standard_lasso =
           Pipeline(steps=[('scaler', scaler),
                           ('lasso', lasso)])
```

Chúng ta cần tạo một lưới các giá trị cho λ . Theo thông lệ, chúng ta chọn một lưới gồm 100 giá trị của λ , phân bố đều trên thang logarit từ lam_max xuống đến $0,01 * \text{lam_max}$. Ở đây, lam_max là giá trị nhỏ nhất của λ có nghiệm bằng 0. Giá trị này bằng tích vô hướng tuyệt đối lớn nhất giữa bất kỳ biến dự đoán nào và biến phản hồi (đã được căn giữa).²⁴

```
Trong [15]: X_s = scaler.fit_transform(X_train)
           n = X_s.shape[0]
           lam_max = np.fabs(X_s.T.dot(Y_train - Y_train.mean())) .max() / n
           param_grid = {'alpha': np.exp(np.linspace(0, np.log(0.01), 100)) * lam_max}
```

Lưu ý rằng chúng ta phải biến đổi dữ liệu trước, vì thang đo của các biến ảnh hưởng đến việc lựa chọn λ . Bây giờ chúng ta thực hiện kiểm định chéo bằng cách sử dụng chuỗi giá trị λ này.

```
Trong [16]: cv = KFold(10,
                     shuffle=True,
                     random_state=1)
           grid = GridSearchCV(lasso,
```

24. Việc chứng minh kết quả này nằm ngoài phạm vi của cuốn sách này.

```

        tham số_lưới,
        cv=cv,
        scoring='neg_mean_absolute_error') grid.fit(X_train,
Y_train);

```

Chúng tôi trích xuất mô hình lasso có sai số tuyệt đối trung bình tốt nhất được kiểm định chéo, và đánh giá hiệu suất của nó trên `X_test` và `Y_test`, những tập dữ liệu không được sử dụng trong kiểm định chéo.

```

Trong [17]: trained_lasso = grid.best_estimator_
Yhat_test = trained_lasso.predict(X_test)
np.fabs(Yhat_test - Y_test).mean()

```

Ra[17]: 257.2382

Kết quả này tương tự với kết quả chúng ta thu được đối với mô hình tuyến tính được hiệu chỉnh bằng phương pháp bình phương tối thiểu. Tuy nhiên, kết quả có thể thay đổi rất nhiều tùy thuộc vào cách chia tập dữ liệu huấn luyện/kiểm tra; chúng tôi khuyến khích người đọc thử một seed khác trong khối mã 12 và chạy lại đoạn mã tiếp theo cho đến thời điểm này.

Xác định cấu trúc mạng: Lớp và tính kế thừa

Để xây dựng mạng nơ-ron, trước tiên chúng ta thiết lập cấu trúc mô hình mô tả mạng. Việc này đòi hỏi chúng ta phải định nghĩa các lớp mới dành riêng cho mô hình mà chúng ta muốn xây dựng. Thông thường, trong `PyTorch`, điều này được thực hiện bằng cách kế thừa một lớp từ biểu diễn chung của mạng, và đó là cách tiếp cận mà chúng ta sử dụng ở đây. Mặc dù ví dụ này khá đơn giản, chúng ta sẽ đi qua các bước một cách chi tiết, vì nó sẽ rất hữu ích cho các ví dụ phức tạp hơn sau này.

```

Trong [18]: class HittersModel(nn.Module):

    def __init__(self, input_size):
        super(HittersModel, self).__init__() self.flatten
        = nn.Flatten() self.sequential =
        nn.Sequential( nn.Linear(input_size, 50),
            nn.ReLU(), nn.Dropout(0.4),
            nn.Linear(50,
                1))

    def forward(self, x):
        x = self.flatten(x)
        return torch.flatten(self.sequential(x))

```

Câu lệnh `'class'` xác định đoạn mã này là một khai báo cho lớp `'HittersModel'` kế thừa từ lớp cơ sở `'nn.Module'`. Lớp cơ sở này phổ biến trong `torch` và đại diện cho các ánh xạ trong mạng nơ-ron.

Nằm thụt vào bên dưới phần khai báo lớp là các phương thức của lớp này: trong trường hợp này là `__init__` và `forward`. Phương thức `__init__` được gọi khi một thể hiện của lớp được tạo ra như trong ô bên dưới. Trong các phương thức, `self` luôn tham chiếu đến một thể hiện của lớp. Trong phương thức `__init__`, chúng ta đã gắn hai đối tượng vào `self` dưới dạng thuộc tính: `flatten` và `sequential`. Chúng được sử dụng trong phương thức `forward` để mô tả bản đồ mà mô-đun này triển khai.

Có thêm một dòng nữa trong phương thức `__init__`, đó là một lời gọi đến `super()`. Hàm này cho phép các lớp con (ví dụ: `HittersModel`) truy cập các phương thức của lớp mà chúng kế thừa. Ví dụ, lớp `nn.Module` có phương thức của nó `siêu()`. Phương thức `__init__` riêng, khác với `HittersModel.__init__()` phương thức mà chúng ta đã viết ở trên. Việc sử dụng `super()` cho phép chúng ta gọi phương thức đó của lớp cơ sở. Đối với các mô hình `torch`, chúng ta sẽ luôn thực hiện điều này bằng `super()`. Gọi hàm này vì nó cần thiết để mô hình được `Torch` diễn giải đúng cách.

Đối tượng `nn.Module` có nhiều phương thức hơn là chỉ `__init__` và `forward`. Các phương pháp này có thể truy cập trực tiếp bởi các thể hiện của `HittersModel` vì sự kế thừa này. Một trong những phương pháp như vậy mà chúng ta sẽ sớm thấy là phương thức `eval()`, Được sử dụng để vô hiệu hóa dropout khi chúng ta muốn đánh giá mô hình trong quá trình kiểm thử dữ liệu.

```
Trong [19]: hit_model = HittersModel(X.shape[1])
```

Đối tượng `self.sequential` là sự kết hợp của bốn bản đồ. Bản đồ đầu tiên ánh xạ 19 đặc điểm của `Hitters` sang 50 chiều, giới thiệu $50 \times 19 + 50$ Các tham số cho trọng số và hệ số chặn của bản đồ (thường được gọi là độ lệch). Lớp này sau đó được ánh xạ tới một lớp ReLU, tiếp theo là một lớp dropout 40%. và cuối cùng là một phép ánh xạ tuyến tính xuống còn 1 chiều, cũng có độ lệch. Tổng cộng Do đó, số lượng tham số có thể huấn luyện là $50 \times 19 + 50 + 50 + 1 = 1051$.

Gói `torchinfo` cung cấp hàm `summary()` giúp tóm tắt thông tin này một cách gọn gàng. Chúng ta chỉ định kích thước của dữ liệu đầu vào và xem kích thước của mỗi tensor khi nó đi qua các lớp của mạng.

```
Trong [20]: tóm tắt(mô hình tấn công,
                input_size=X_train.shape,
                col_names=['input_size',
                           'kích_cỡ_đầu_ra',
                           'num_params'])
```

Ra[20]: =====

Lớp (loại: chỉ số độ sâu)	Hình dạng đầu vào	Hình dạng đầu ra	Tham số #
=====			
Mô hình người đánh bóng	[175, 19]	[175]	--
Làm phẳng: 1-1	[175, 19]	[175, 19]	--
Theo trình tự: 1-2	[175, 19]	[175, 1]	--
Tuyến tính: 2-1	[175, 19]	[175, 50]	1.000
ReLU: 2-2	[175, 50]	[175, 50]	--
Tỷ lệ bỏ học: 2-3	[175, 50]	[175, 50]	--
Tuyến tính: 2-4	[175, 50]	[175, 1]	51
=====			
Tổng số tham số: 1.051			
Số tham số có thể huấn luyện: 1.051			

Chúng tôi đã cắt bớt phần cuối của kết quả đầu ra một chút, ở đây và trong các phần tiếp theo. công dụng.

Giờ chúng ta cần chuyển đổi dữ liệu huấn luyện của mình sang dạng có thể truy cập được. `torch`. Kiểu dữ liệu cơ bản trong `torch` là `tensor`, rất giống với một mảng `ndarray` từ các chương đầu. Chúng tôi cũng lưu ý ở đây rằng `torch` thường Hoạt động với dữ liệu 32-bit (độ chính xác đơn) thay vì 64-bit (độ chính xác kép). số thập phân. Do đó, chúng ta chuyển đổi dữ liệu của mình sang `np.float32` trước khi thực hiện. tạo thành `tensor`. Sau đó, các `tensor` X và Y được sắp xếp thành một tập dữ liệu.

Tập dữ liệu

Được nhận dạng bởi `torch` bằng cách sử dụng `TensorDataset()`.

Tensor

Tập dữ liệu()

```
Trong [21]: X_train_t = torch.tensor(X_train.astype(np.float32))
           Y_train_t = torch.tensor(Y_train.astype(np.float32)) hit_train
           = TensorDataset(X_train_t, Y_train_t)
```

Chúng tôi cũng làm tương tự với dữ liệu thử nghiệm.

```
Trong [22]: X_test_t = torch.tensor(X_test.astype(np.float32))
           Y_test_t = torch.tensor(Y_test.astype(np.float32)) hit_test
           = TensorDataset(X_test_t, Y_test_t)
```

Cuối cùng, tập dữ liệu này được chuyển đến `DataLoader()` để chuyển dữ liệu vào mạng của chúng ta. Mặc dù điều này có vẻ như tốn nhiều tài nguyên, cấu trúc này rất hữu ích cho các tác vụ phức tạp hơn, nơi dữ liệu có thể nằm trên các máy khác nhau hoặc nơi dữ liệu phải được chuyển đến GPU. Chúng tôi cung cấp một hàm trợ giúp `SimpleDataModule()` trong ISLP để đơn giản hóa tác vụ này cho việc sử dụng thông thường. Một trong các đối số của nó là `num_workers`, cho biết chúng ta sẽ sử dụng bao nhiêu tiến trình để tải dữ liệu. Đối với dữ liệu nhỏ như `Hitters`, điều này sẽ có ít ảnh hưởng, nhưng nó mang lại lợi thế cho các ví dụ `MNIST` và `CIFAR100` bên dưới. Gói `torch` sẽ kiểm tra tiến trình đang chạy và xác định số lượng worker tối đa. Chúng tôi đã bao gồm một hàm `rec_num_workers()` để tính toán điều này để biết số lượng worker hợp lý (ở đây tối đa là 16).

Dữ liệu đơn giản
Module()

```
Trong [23]: max_num_workers = rec_num_workers()
```

Cấu hình huấn luyện chung trong `pytorch_lightning` bao gồm dữ liệu huấn luyện, dữ liệu xác thực và dữ liệu kiểm thử. Mỗi loại dữ liệu này được thể hiện bằng các trình tải dữ liệu khác nhau. Trong mỗi chu kỳ huấn luyện, chúng ta thực hiện một bước huấn luyện để học mô hình và một bước kiểm định để theo dõi lỗi. Dữ liệu kiểm thử thường được sử dụng vào cuối quá trình huấn luyện để đánh giá mô hình.

Trong trường hợp này, vì chúng ta chỉ chia dữ liệu thành tập huấn luyện và tập kiểm thử, chúng ta sẽ sử dụng dữ liệu kiểm thử làm dữ liệu xác thực với tham số `'validation=hit_test'`. Tham số `'validation'` có thể là một số thực nằm giữa 0 và 1, một số nguyên hoặc một đối tượng `'Dataset'`. Nếu là số thực (tương ứng là số nguyên), nó được hiểu là tỷ lệ phần trăm (tương ứng là số lượng) các quan sát trong tập huấn luyện được sử dụng để xác thực. Nếu đó là một `Dataset`, nó sẽ được chuyển trực tiếp đến trình tải dữ liệu.

```
Trong [24]: hit_dm = SimpleDataModule(hit_train,
                                     hit_test,
                                     batch_size=32,
                                     num_workers=min(4, max_num_workers),
                                     validation=hit_test)
```

Tiếp theo, chúng ta cần cung cấp một module `pytorch_lightning` để điều khiển các bước được thực hiện trong quá trình huấn luyện. Chúng ta cung cấp các phương thức cho `SimpleModule()` để ghi lại giá trị của hàm mất mát và bất kỳ số liệu bổ sung nào khác vào cuối mỗi epoch. Các thao tác này được điều khiển bởi các phương thức `SimpleModule.[training/test/validation]_step()`, mặc dù chúng ta sẽ không sửa đổi chúng trong các ví dụ của mình.

25 Điều này phụ thuộc vào phần cứng máy tính và số lượng lỗi xử lý có sẵn.


```
Trong [25]: hit_module = SimpleModule.regression(hit_model,
                                                metrics={'mae':MeanAbsoluteError()})
```

Bằng cách sử dụng phương thức `SimpleModule.regression()` , chúng tôi chỉ ra rằng chúng tôi sẽ sử dụng tổn thất bình phương sai số như trong (10.23).Chúng tôi cũng yêu cầu theo dõi sai số tuyệt đối trung bình trong các số liệu được ghi lại.

`SimpleModule.regression`

Chúng tôi ghi lại kết quả thông qua `CSVLogger()`, trong trường hợp này, hàm này lưu trữ các kết quả. trong một tệp CSV nằm trong thư mục `logs/hitters`. Sau khi quá trình điều chỉnh hoàn tất, Điều này cho phép chúng ta tải kết quả dưới dạng `pd.DataFrame()` và trực quan hóa chúng. Dưới đây là một số cách để ghi lại kết quả trong `pytorch_lightning`. Tuy nhiên, chúng ta sẽ không đi sâu vào chi tiết những vấn đề đó ở đây.

```
Trong [26]: hit_logger = CSVLogger('logs', name='hitters')
```

Cuối cùng, chúng ta đã sẵn sàng huấn luyện mô hình và ghi lại kết quả. Chúng ta sử dụng Sử dụng đối tượng `Trainer()` từ thư viện `pytorch_lightning` để thực hiện công việc này. Tham số `datamodule=hit_dm`` cho biết cách xử lý nhật ký huấn luyện/kiểm tra/thử nghiệm. được tạo ra, trong khi đối số đầu tiên `hit_module` chỉ định mạng. kiến trúc cũng như các bước huấn luyện/xác thực/kiểm thử. Các hàm gọi lại (callbacks) Lập luận cho phép thực hiện nhiều nhiệm vụ tại các thời điểm khác nhau trong khi huấn luyện một mô hình. Tại đây, hàm gọi lại `ErrorTracker()` của chúng ta sẽ cho phép tính toán lỗi xác thực trong quá trình huấn luyện và cuối cùng là lỗi kiểm thử. Bây giờ chúng ta tiến hành điều chỉnh mô hình. Mô hình cho 50 kỳ nguyên.

```
Trong [27]: hit_trainer = Trainer(deterministic=True,
                                max_epochs=50,
                                log_every_n_steps=5,
                                logger=hit_logger,
                                hàm gọi lại=[ErrorTracker()])
hit_trainer.fit(hit_module, datamodule=hit_dm)
```

Ở mỗi bước của SGD, thuật toán chọn ngẫu nhiên 32 quan sát huấn luyện để tính toán độ dốc. Hãy nhớ lại từ Phần 10.7 rằng một epoch tương ứng với số bước SGD cần thiết để xử lý n quan sát. Vì tập huấn luyện có n = 175 và chúng ta đã chỉ định `batch_size` là Trong quá trình xây dựng `hit_dm`, một epoch tương ứng với $175/32 = 5,5$ bước SGD. Sau khi đã hiệu chỉnh mô hình, chúng ta có thể đánh giá hiệu suất trên dữ liệu thử nghiệm. sử dụng phương thức `test()` của trình huấn luyện.

```
Trong [28]: hit_trainer.test(hit_module, datamodule=hit_dm)
```

Out[28]: [{'test_loss': 104098.5469, 'test_mae': 229.5012}]

Kết quả của quá trình khớp dữ liệu đã được ghi vào một tệp CSV. Chúng ta có thể tìm thấy kết quả cụ thể cho lần chạy này trong thuộc tính `experiment.metrics_file_path` của trình ghi nhật ký của chúng tôi. Lưu ý rằng mỗi khi mô hình được huấn luyện, trình ghi nhật ký sẽ xuất ra kết quả. Kết quả được tạo thành một thư mục con mới trong thư mục `logs/hitters` của chúng ta. Bây giờ chúng ta sẽ tạo một đồ thị biểu diễn MAE (sai số tuyệt đối trung bình) theo hàm số của số lượng kỷ nguyên. Đầu tiên, chúng ta truy xuất các bản tóm tắt đã được ghi lại.

```
hit_results = pd.read_csv(hit_logger.experiment.metrics_file_path)
```

Vì chúng ta sẽ tạo ra các đồ thị tương tự trong các ví dụ sau, nên chúng ta viết một đoạn mã đơn giản. Hàm chung để tạo ra biểu đồ này.

```

Trong [29]: def summary_plot(results,
                        riu,
                        col='loss',
                        valid_legend='Xác thực',
                        training_legend='Huấn luyện', ylabel='Loss',
                        fontsize=20):

    cho (cột,
         color,
         label) in zip([f'train_{col}_epoch', f'valid_{col}'], ['black',
                                                                'red'], [training_legend,
                                                                valid_legend]):

        results.plot(x='epoch',
                     y=column, label=label,
                     marker='o', color=color, ax=ax)

        ax.set_xlabel('Epoch')
        ax.set_ylabel(ylabel)
        return ax

```

Bây giờ chúng ta thiết lập các trục tọa độ và sử dụng hàm của mình để tạo ra biểu đồ MAE.

```

Trong [30]: fig, ax = subplots(1, 1, figsize=(6, 6))
ax = summary_plot(hit_results,
                  riu,
                  col='mae',
                  ylabel='MAE',
                  valid_legend='Xác thực (=Kiểm thử)')
ax.set_ylim([0, 400])
ax.set_xticks(np.linspace(0, 50, 11).astype(int));

```

Chúng ta có thể dự đoán trực tiếp từ mô hình cuối cùng và đánh giá hiệu suất của nó trên dữ liệu kiểm thử. Trước khi huấn luyện, chúng ta gọi phương thức `'eval()'` của `'hit_model'`. Điều này cho `torch` biết rằng mô hình này đã được huấn luyện thành công, để chúng ta có thể sử dụng nó để dự đoán trên dữ liệu mới. Đối với mô hình của chúng ta ở đây, thay đổi lớn nhất là các lớp dropout sẽ bị tắt, tức là không có trọng số nào bị loại bỏ ngẫu nhiên khi dự đoán trên dữ liệu mới.

```

Trong [31]: hit_model.eval()
preds = hit_module(X_test_t)
torch.abs(Y_test_t - preds).mean()

```

```
Out[31]: tensor(229.5012, grad_fn=<MeanBackward0 >)
```

Dọn dẹp

Trong quá trình thiết lập mô-đun dữ liệu, chúng tôi đã khởi tạo một số tiến trình worker sẽ tiếp tục chạy. Chúng tôi xóa tất cả các tham chiếu đến các đối tượng `torch` để đảm bảo các tiến trình này sẽ bị chấm dứt.

```
Trong [32]: del(Hitters,
             hit_model, hit_dm,
             hit_logger,
             hit_test, hit_train,
             X, Y,
             X_test, X_train,
             Y_test, Y_train,
             X_test_t, Y_test_t,
             hit_trainer, hit_module)
```

10.9.2 Mạng đa lớp trên dữ liệu chữ số MNIST

Gói `torchvision` đi kèm với một số bộ dữ liệu ví dụ, bao gồm cả dữ liệu chữ số `MNIST` . Bước đầu tiên của chúng ta là tải xuống các bộ dữ liệu huấn luyện và kiểm tra; hàm `MNIST()` trong `torchvision.datasets` được cung cấp cho mục đích này. Dữ liệu sẽ được tải xuống lần đầu tiên khi hàm này được thực thi và được lưu trữ trong thư mục `data/MNIST`.

```
Trong [33]: (mnist_train,
            mnist_test) = [MNIST(root='data', train=train,
                                download=True,
                                transform=ToTensor())
                           for train in [True, False]]

mnist_train
```

```
Out[33]: Tập dữ liệu MNIST
Số lượng điểm dữ liệu: 60000
Vị trí thư mục gốc: dữ liệu
Chia tách: Tầu hỏa
Biến đổi tiêu chuẩn
Biến đổi: ToTensor()
```

Tập dữ liệu huấn luyện gồm 60.000 hình ảnh và tập dữ liệu kiểm tra gồm 10.000 hình ảnh. Các hình ảnh có kích thước 28 × 28 và được lưu trữ dưới dạng ma trận điểm ảnh. Chúng ta cần chuyển đổi mỗi điểm ảnh thành một vectơ.

Mạng nơ-ron khá nhạy cảm với thang đo của đầu vào, tương tự như phương pháp điều chỉnh ridge và lasso bị ảnh hưởng bởi việc thay đổi tỷ lệ. Ở đây, đầu vào là các giá trị thang độ xám tám bit nằm giữa 0 và 255, vì vậy chúng ta điều chỉnh tỷ lệ về khoảng đơn vị. ^{>26} Phép biến đổi này, cùng với một số sắp xếp lại trực, được thực hiện bởi phép biến đổi `ToTensor()` từ gói `torchvision.transforms` .</sup>

Như trong ví dụ về `Hitters` , chúng ta tạo ra một mô-đun dữ liệu từ quá trình huấn luyện và các tập dữ liệu thử nghiệm, trong đó 20% hình ảnh huấn luyện được dành riêng cho việc xác thực.

```
Trong [34]: mnist_dm = SimpleDataModule(mnist_train, mnist_test,
                                       validation=0.2,

                                       num_workers=max_num_workers, batch_size=256)
```

26Lưu ý: tám bit nghĩa là 28, bằng 256. Vì quy ước là bắt đầu từ 0, Các giá trị có thể nằm trong khoảng từ 0 đến 255.

Hãy cùng xem dữ liệu sẽ được đưa vào mạng của chúng ta. Chúng ta sẽ lặp qua một vài phần đầu tiên của tập dữ liệu thử nghiệm, dừng lại sau 2 lô:

```
Trong [35]: for idx, (X_, Y_) in enumerate(mnist_dm.train_dataloader()): print('X: ',
        X_.shape) print('Y: ',
        Y_.shape)
        nếu idx >= 1:
            phá vỡ
```

```
X: torch.Size([256, 1, 28, 28])
Y: torch.Size([256])
X: torch.Size([256, 1, 28, 28])
Y: torch.Size([256])
```

Ta thấy rằng tập dữ liệu X cho mỗi lô bao gồm 256 hình ảnh có kích thước **1x28x28**. Ở đây, số 1 biểu thị một kênh đơn (ảnh xám). Đối với ảnh RGB như **CIFAR100** bên dưới, ta sẽ thấy rằng số 1 trong kích thước sẽ được thay thế bằng số 3 để biểu thị ba kênh RGB.

Giờ chúng ta đã sẵn sàng để xác định cấu hình cho mạng nơ-ron của mình.

```
Trong [36]: class MNISTModel(nn.Module):
        def __init__(self):
            super(MNISTModel, self).__init__()
            self.layer1 =

                nn.Sequential( nn.Flatten(),

                nn.Linear(28*28,
                256), nn.ReLU(), nn.Dropout(0.4))
            self.layer2 =

                nn.Sequential( nn.Linear(256, 128),
                nn.ReLU(),
                nn.Dropout(0.3))
            self._forward = nn.Sequential( self.layer1, self.layer2, nn.Linear(128, 10))
        def forward(self, x):
            return self._forward(x)
```

Như ta thấy ở lớp đầu tiên, mỗi ảnh **1x28x28** được làm phẳng, sau đó được ánh xạ xuống 256 chiều, nơi ta áp dụng hàm kích hoạt ReLU với tỷ lệ dropout 40%. Lớp thứ hai ánh xạ đầu ra của lớp đầu tiên xuống 128 chiều, áp dụng hàm kích hoạt ReLU với tỷ lệ dropout 30%. Cuối cùng, 128 chiều được ánh xạ xuống 10, bằng số lượng lớp trong tập dữ liệu **MNIST**.

```
Trong [37]: mnist_model = MNISTModel()
```

Chúng ta có thể kiểm tra xem mô hình có tạo ra đầu ra với kích thước như mong đợi dựa trên lô dữ liệu **X_** hiện có ở trên hay không.

```
Trong [38]: mnist_model(X_).size()
```

```
Out[38]: torch.Size([256, 10])
```

Hãy cùng xem tóm tắt mô hình. Thay vì **input_size**, chúng ta có thể truyền vào một tensor có hình dạng chính xác. Trong trường hợp này, chúng ta truyền vào **X_** đã được xử lý theo lô ở trên.

```
Trong [39]: tóm tắt(mnist_model,
                input_data=X_,
                col_names=['input_size',
                           'kích_cỡ_đầu_ra',
                           'num_params'])
```

Ra[39]: =====

Lớp (loại: chỉ số độ sâu)	Hình dạng đầu vào	Hình dạng đầu ra	Tham số #		
=====					
Mô hình MNIST	[256, 1, 28, 28]	[256, 10]	[256, 1, --		
Tuần tự: 1-1	28, 28]	[256, 10]	[256, 1, 28, 28]	--	
Tuần tự: 2-1	[256, 256]	[256, 1, 28, 28]	[256, 784]	--	
Làm phẳng: 3-1	[256, 784]	[256, 256]	[256, 256]	[256, --	
Tuyến tính: 3-2	256]	[256, 256]	[256, 256]	[256, 256]	200.960
ReLU: 3-3	[256, 128]	[256, 256]	[256, 128]	[256, --	
Tỷ lệ bỏ học: 3-4	128]	[256, 128]	[256, 128]	[256, 128]	--
Tuần tự: 2-2	[256, 10]				--
Tuyến tính: 3-5					32.896
ReLU: 3-6					--
Tỷ lệ bỏ học: 3-7					--
Tuyến tính: 2-3					1.290
=====					
Tổng số tham số: 235.146					
Số tham số có thể huấn luyện: 235.146					

Sau khi đã thiết lập cả mô hình và mô-đun dữ liệu, việc hiệu chỉnh mô hình này là...
giờ gần như giống hệt ví dụ về `Hitters` . Trái ngược với phép hồi quy của chúng ta
Trong mô hình này, chúng ta sẽ sử dụng phương thức `SimpleModule.classification()` sử dụng hàm mất mát
entropy chéo thay vì sai số bình phương trung bình.

`SimpleModule.
phân loại()`

```
Trong [40]: mnist_module = SimpleModule.classification(mnist_model)
mnist_logger = CSVLogger('logs', name='MNIST')
```

Giờ chúng ta đã sẵn sàng. Bước cuối cùng là cung cấp dữ liệu huấn luyện, và
Phù hợp với mô hình.

```
Trong [41]: mnist_trainer = Trainer(deterministic=True,
                                    max_epochs=30,
                                    logger=mnist_logger,
                                    hàm gọi lại=[ErrorTracker()])

mnist_trainer.fit(mnist_module,
                  datamodule=mnist_dm)
```

Chúng tôi đã ẩn phần đầu ra ở đây, đây là báo cáo tiến độ về...
Việc khớp mô hình, được nhóm theo từng kỷ nguyên. Điều này rất hữu ích, vì trên quy mô lớn
Việc khớp dữ liệu có thể mất thời gian. Quá trình khớp mô hình này mất 245 giây trên một tập dữ liệu.
MacBook Pro với chip Apple M1 Pro 10 nhân và RAM 16 GB.
RAM. Ở đây chúng ta đã chỉ định tỷ lệ phân chia dữ liệu xác thực là 20%, vì vậy dữ liệu huấn luyện thực tế là...
Đã thực hiện trên 80% trong số 60.000 quan sát trong tập dữ liệu huấn luyện. Đây là một
một phương án thay thế cho việc thực sự cung cấp dữ liệu xác thực, giống như chúng tôi đã làm cho đội `Hitters`.
dữ liệu. SGD sử dụng các lô gồm 256 quan sát để tính toán độ dốc, và
Khi thực hiện phép tính, ta thấy rằng một kỷ nguyên tương ứng với 188 bậc gradient.
các bước.

`SimpleModule.classification()` bao gồm một chỉ số độ chính xác theo mặc định.
Các chỉ số phân loại khác có thể được thêm vào từ `torchmetrics`. Chúng ta sẽ sử dụng
Chúng tôi sử dụng hàm `summary_plot()` để hiển thị độ chính xác qua các kỷ nguyên.

```
Trong [42]: mnist_results = pd.read_csv(mnist_logger.experiment.metrics_file_path)

fig, ax = subplots(1, 1, figsize=(6, 6))
summary_plot(mnist_results,
              riu,
              col='accuracy',
              ylabel='Accuracy')
ax.set_ylim([0.5, 1])
ax.set_ylabel('Độ chính xác')
ax.set_xticks(np.linspace(0, 30, 7).astype(int));
```

Một lần nữa, chúng ta đánh giá độ chính xác bằng cách sử dụng phương thức `test()` của bộ huấn luyện. Mô hình này đạt độ chính xác 97% trên dữ liệu kiểm thử.

```
Trong [43]: mnist_trainer.test(mnist_module,
                             datamodule=mnist_dm)
```

```
Out[43]: [{'test_loss': 0.1471, 'test_accuracy': 0.9681}]
```

Bảng 10.1 cũng trình bày tỷ lệ lỗi phát sinh từ LDA (Chương 4) và hồi quy logistic đa lớp. Đối với LDA, chúng tôi mời người đọc tham khảo Mục 4.7.3.

Mặc dù chúng ta có thể sử dụng hàm `LogisticRegression()` của `sklearn` để huấn luyện hồi quy logistic đa lớp, nhưng ở đây chúng ta thiết lập để huấn luyện mô hình đó bằng `torch`. Chúng ta chỉ có một lớp đầu vào và một lớp đầu ra, và bỏ qua các lớp ẩn!

```
Trong [44]: class MNIST_MLR(nn.Module): def
            __init__(self):
                super(MNIST_MLR, self).__init__()
                self.linear = nn.Sequential(nn.Flatten(),
                                             nn.Linear(784, 10))

            def forward(self, x):
                trả về self.linear(x)

mlr_model = MNIST_MLR()
mlr_module = SimpleModule.classification(mlr_model) mlr_logger =
CSVLogger('logs', name='MNIST_MLR')
```

```
Trong [45]: mlr_trainer = Trainer(deterministic=True,
                                max_epochs=30,
                                callbacks=[ErrorTracker()])
mlr_trainer.fit(mlr_module, datamodule=mnist_dm)
```

Chúng tôi điều chỉnh mô hình như trước và tính toán kết quả kiểm tra.

```
Trong [46]: mlr_trainer.test(mlr_module,
                             datamodule=mnist_dm)
```

```
Out[46]: [{'test_loss': 0.3187, 'test_accuracy': 0.9241}]
```

Độ chính xác đạt trên 90% ngay cả với mô hình khá đơn giản này.

Tương tự như ví dụ về `Hitters`, chúng ta xóa một số đối tượng đã tạo ở trên.

```
Trong [47]: del(mnist_test,
               mnist_train,
```

```
mnist_model,
mnist_dm,
mnist_trainer,
mnist_module,
mnist_results,
mlr_model,
mlr_module,
mlr_trainer)
```

10.9.3 Mạng nơ-ron tích chập

Trong phần này, chúng ta sẽ huấn luyện một mạng CNN cho dữ liệu **CIFAR100**, có sẵn trong gói **torchvision**. Dữ liệu này được sắp xếp theo cách tương tự như dữ liệu **MNIST**.

```
Trong [48]: (cifar_train,
            cifar_test) = [CIFAR100(root="data", train=train,
                                   download=True)

                           cho tàu hỏa ở [Đúng, Sai]]
```

```
Trong [49]: transform = ToTensor()
cifar_train_X = torch.stack([transform(x) for x in cifar_train.data])
cifar_test_X =
torch.stack([transform(x) for x in cifar_test.data]) cifar_train
=
TensorDataset(cifar_train_X,
               torch.tensor(cifar_train.targets))
cifar_test = TensorDataset(cifar_test_X,
                           torch.tensor(cifar_test.targets))
```

Tập dữ liệu **CIFAR100** bao gồm 50.000 hình ảnh huấn luyện, mỗi hình ảnh được biểu diễn bằng một tensor ba chiều: mỗi hình ảnh ba màu được biểu diễn dưới dạng một tập hợp ba kênh, mỗi kênh bao gồm 32×32 pixel tám bit. Chúng tôi chuẩn hóa như đã làm với các chữ số, nhưng vẫn giữ nguyên cấu trúc mảng. Điều này được thực hiện bằng phép biến đổi **ToTensor()**.

Việc tạo mô-đun dữ liệu tương tự như ví dụ **MNIST**.

```
Trong [50]: cifar_dm = SimpleDataModule(cifar_train, cifar_test,
                                       validation=0.2,

                                       num_workers=max_num_workers,
                                       batch_size=128)
```

Chúng ta lại xem xét hình dạng của các lô dữ liệu điển hình trong trình tải dữ liệu của mình.

```
Trong [51]: for idx, (X_, Y_) in enumerate(cifar_dm.train_dataloader()): print('X: ', X_.shape) print('Y:
            ', Y_.shape) if idx >= 1:

            phá vỡ
```

```
X: torch.Size([128, 3, 32, 32])
Y: torch.Size([128])
X: torch.Size([128, 3, 32, 32])
Y: torch.Size([128])
```

Trước khi bắt đầu, chúng ta hãy xem xét một số hình ảnh huấn luyện; đoạn mã tương tự đã tạo ra Hình 10.5 trên trang 406. Ví dụ bên dưới cũng minh họa rằng các đối tượng `TensorDataset` có thể được lập chỉ mục bằng số nguyên – chúng ta đang chọn ngẫu nhiên các hình ảnh từ dữ liệu huấn luyện bằng cách lập chỉ mục `cifar_train`. Để hiển thị chính xác, chúng ta phải sắp xếp lại thứ tự các chiều bằng cách gọi hàm `np.transpose()`.

```
Trong [52]: fig, axes = subplots(5, 5, figsize=(10,10)) rng =
np.random.default_rng(4) indices =
rng.choice(np.arange(len(cifar_train)), 25,
           replace=False).reshape((5,5))

for i in range(5): for
    j in range(5): idx =
        indices[i,j]
        axes[i,j].imshow(np.transpose(cifar_train[idx][0], [1,2,0]),
                           interpolation=None)

        axes[i,j].set_xticks([])
        axes[i,j].set_yticks([])
```

Ở đây, phương thức `imshow()` nhận biết từ hình dạng của đối số rằng đó là một mảng 3 chiều, với `.imshow()` chiều cuối cùng là chỉ số của ba kênh màu RGB.

Chúng tôi sử dụng một mạng CNN có kích thước vừa phải để minh họa, tương tự về cấu trúc với Hình 10.8. Chúng tôi sử dụng một số lớp, mỗi lớp bao gồm các bước tích chập, hàm ReLU và gộp tối đa. Đầu tiên, chúng tôi định nghĩa một module xác định một trong các lớp này. Giống như trong các ví dụ trước, chúng tôi ghi đè lên các phương thức `__init__()` và `forward()` của `nn.Module`. Module do người dùng định nghĩa này giờ đây có thể được sử dụng theo nhiều cách tương tự như `nn.Linear()` hoặc `nn.Dropout()`.

```
Trong [53]: class BuildingBlock(nn.Module):

    def __init__(self,
                  in_channels,
                  out_channels):

        super(BuildingBlock, self).__init__()
        self.conv = nn.Conv2d(in_channels=in_channels,
                               out_channels=out_channels,
                               kernel_size=(3,3),
                               padding='same')

        self.activation = nn.ReLU()
        self.pool = nn.MaxPool2d(kernel_size=(2,2))

    def forward(self, x):
        return self.pool(self.activation(self.conv(x)))
```

Lưu ý rằng chúng ta đã sử dụng đối số `padding = "same"` cho hàm `nn.Conv2d()`, điều này đảm bảo rằng các kênh đầu ra có cùng kích thước với các kênh đầu vào. Có 32 kênh trong lớp ẩn đầu tiên, trái ngược với ba kênh trong lớp đầu vào. Chúng ta sử dụng bộ lọc tích chập 3×3 cho mỗi kênh trong tất cả các lớp. Mỗi phép tích chập được theo sau bởi một lớp gộp tối đa trên các khối 2×2 .

Trong quá trình xây dựng mô hình học sâu cho dữ liệu CIFAR100, chúng tôi sử dụng một số mô-đun `BuildingBlock()` của mình theo trình tự. Ví dụ đơn giản này minh họa một số sức mạnh của `torch`. Người dùng có thể tự định nghĩa các mô-đun của riêng mình.

450 10. Học sâu

có thể được kết hợp trong các mô-đun khác. Cuối cùng, mọi thứ đều được sắp xếp bởi một huấn luyện viên thông thường.

```
Trong [54]: class CIFARModel(nn.Module):

    def __init__(self):
        super(CIFARModel, self).__init__()
        kích thước = [(3,32),
                        (32,64),
                        (64,128),
                        (128,256)]

        self.conv = nn.Sequential(*[BuildingBlock(in_, out_)
                                     cho kích thước đầu vào, đầu ra])

        self.output = nn.Sequential(nn.Dropout(0.5),
                                     nn.Linear(2*2*256, 512),
                                     nn.ReLU(),
                                     nn.Linear(512, 100))

    def forward(self, x):
        val = self.conv(x)
        val = torch.flatten(val, start_dim=1)
        trả về self.output(val)
```

Chúng tôi xây dựng mô hình và xem xét bản tóm tắt. (Chúng tôi đã tạo ra các ví dụ (của X_ trước đó.)

```
Trong [55]: cifar_model = CIFARModel()

tóm tắt(cifar_model,
        input_data=X_,
        col_names=['input_size',
                  'kích_cỡ_đầu_ra',
                  'num_params'])
```

Out[55]:	=====				
	Hình dạng đầu vào của lớp (loại: độ sâu-chỉ số)	Hình dạng đầu ra	Tham số	#	
	=====	=====			
	Mô hình CIFAR	[128, 3, 32, 32] [128, 100]	[128, 3,	--	
	Tuần tự: 1-1	32, 32] [128, 256, 2, 2]		--	
	Khối xây dựng: 2-1	[128, 3, 32, 32] [128, 32, 16, 16]	--		
	Conv2d: 3-1	[128, 3, 32, 32] [128, 32, 32, 32]	896		
	ReLU: 3-2	[128, 32, 32, 32] [128, 32, 32, 32]	--		
	MaxPool2d: 3-3	[128, 32, 32, 32] [128, 32, 16, 16]	--		
	Khối xây dựng: 2-2	[128, 32, 16, 16] [128, 64, 8, 8]	[128, 32, 16, 16] [128,	--	
	Conv2d: 3-4	64, 16, 16] 18,496			
	ReLU: 3-5	[128, 64, 16, 16] [128, 64, 16, 16]	--		
	MaxPool2d: 3-6	[128, 64, 16, 16] [128, 64, 8, 8]	--		
	Khối xây dựng: 2-3	[128, 64, 8, 8] [128, 64, 8, 8]	[128, 128, 4, 4]	--	
	Conv2d: 3-7	[128, 128, 8, 8]	[128, 128, 8, 8] 73,856		
	ReLU: 3-8		[128, 128, 8, 8]	--	
	MaxPool2d: 3-9	[128, 128, 8, 8]	[128, 128, 4, 4]	--	
	Khối xây dựng: 2-4	[128, 128, 4, 4]	[128, 256, 2, 2]	--	
	Conv2d: 3-10	[128, 128, 4, 4]	[128, 256, 4, 4] 295,168		
	ReLU: 3-11	[128, 256, 4, 4]	[128, 256, 4, 4]	--	
	MaxPool2d: 3-12	[128, 256, 4, 4] [128, 1024]	[128, 256, 2, 2]	--	
	Theo trình tự: 1-2	[128, 1024]	[128, 100]	--	
	Tỷ lệ bỏ học: 2-5	[128, 1024]	[128, 1024]	--	
	Tuyến tính: 2-6		[128, 512]	524.800	

ReLU: 2-7	[128, 512]	[128, 512]	--
Tuyến tính: 2-8	[128, 512]	[128, 100]	51.300
=====			

Tổng số tham số: 964.516
Số tham số có thể huấn luyện: 964.516

Tổng số tham số có thể huấn luyện là 964.516. Bằng cách nghiên cứu kích thước của các tham số, chúng ta có thể thấy rằng số kênh giảm đi một nửa ở cả hai chiều sau mỗi phép toán max-pooling. Sau phép toán cuối cùng, chúng ta có một lớp với 256 kênh có kích thước 2 × 2. Sau đó, chúng được làm phẳng thành một lớp dense có kích thước 1.024; nói cách khác, mỗi ma trận 2 × 2 được chuyển đổi thành một vectơ 4 chiều và được đặt cạnh nhau trong một lớp. Tiếp theo là một lớp điều chỉnh dropout, sau đó là một lớp dense khác có kích thước 512, và cuối cùng là lớp đầu ra.

Cho đến nay, chúng ta đã sử dụng trình tối ưu hóa mặc định trong SimpleModule(). Với dữ liệu này, các thí nghiệm cho thấy tốc độ học nhỏ hơn cho kết quả tốt hơn so với tốc độ mặc định 0.01. Ở đây, chúng ta sử dụng trình tối ưu hóa tùy chỉnh với tốc độ học là 0.001. Bên cạnh đó, việc ghi nhật ký và huấn luyện tuân theo một mô hình tương tự như các ví dụ trước đây của chúng ta. Trình tối ưu hóa nhận một đối số `params` để thông báo cho trình tối ưu hóa biết các tham số nào được sử dụng trong SGD (phương pháp giảm độ dốc ngẫu nhiên).

Như chúng ta đã thấy trước đó, các tham số của một mô-đun được biểu diễn dưới dạng tensor. Khi truyền các tham số cho trình tối ưu hóa, chúng ta không chỉ đơn thuần truyền các mảng; một phần cấu trúc của đồ thị được mã hóa trong chính các tensor đó.

```
Trong [56]: cifar_optimizer = RMSprop(cifar_model.parameters(), lr=0.001)
           cifar_module = SimpleModule.classification(cifar_model,
                                                       optimizer=cifar_optimizer)
           cifar_logger = CSVLogger('logs', name='CIFAR100')
```

```
Trong [57]: cifar_trainer = Trainer(deterministic=True,
                                   max_epochs=30,
                                   logger=cifar_logger,
                                   callbacks=[ErrorTracker()])
           cifar_trainer.fit(cifar_module,
                             datamodule=cifar_dm)
```

Mô hình này mất 10 phút hoặc hơn để chạy và đạt độ chính xác khoảng 42% trên dữ liệu thử nghiệm. Mặc dù kết quả này không tệ đối với dữ liệu 100 lớp (một bộ phân loại ngẫu nhiên chỉ đạt độ chính xác 1%), nhưng khi tìm kiếm trên mạng, chúng ta thấy các kết quả khoảng 75%. Thông thường, để đạt được kết quả như vậy cần rất nhiều công sức thiết kế kiến trúc, điều chỉnh tham số chính quy và tốn nhiều thời gian.

Hãy cùng xem xét độ chính xác của quá trình xác thực và huấn luyện qua từng epoch.

```
Trong [58]: log_path = cifar_logger.experiment.metrics_file_path
           cifar_results = pd.read_csv(log_path)
           fig, ax = subplots(1, 1, figsize=(6, 6))
           summary_plot(cifar_results, ax, col='accuracy', ylabel='Accuracy')
           ax.set_xticks(np.linspace(0, 10, 6).astype(int))
           ax.set_ylabel('Độ chính xác')
           ax.set_ylim([0, 1]);
```

452 10. Học sâu

Cuối cùng, chúng tôi đánh giá mô hình của mình trên dữ liệu thử nghiệm.

```
Trong [59]: cifar_trainer.test(cifar_module,
                             datamodule=cifar_dm)
```

```
Out[59]: [{'test_loss': 2.4238 'test_accuracy': 0.4206}]
```

Tăng tốc phần cứng

Khi học sâu trở nên phổ biến trong máy học, các nhà sản xuất phần cứng đã tạo ra các thư viện đặc biệt có thể giúp tăng tốc các bước giảm độ dốc.

Ví dụ, các thiết bị Mac OS có chip M1 có thể đã kích hoạt khung lập trình Metal , giúp tăng tốc độ tính toán của `torch` .

Chúng tôi xin trình bày một ví dụ về cách sử dụng gia tốc này.

Những thay đổi chính nằm ở lệnh gọi `Trainer()` cũng như các chỉ số sẽ được đánh giá trên dữ liệu. Các chỉ số này cần được cho biết vị trí của dữ liệu tại thời điểm đánh giá. Điều này được thực hiện bằng cách gọi phương thức `to()` của các chỉ số.

```
Trong [60]: thử:
            đối với tên, số liệu trong cifar_module.metrics.items():
                cifar_module.metrics[name] = metric.to('mps')
            cifar_trainer_mps = Trainer(accelerator='mps',
                                       deterministic=True,
                                       max_epochs=30)
            cifar_trainer_mps.fit(cifar_module,
                                datamodule=cifar_dm)
            cifar_trainer_mps.test(cifar_module, datamodule=cifar_dm)

ngoại trừ:
    bỏ qua
```

Điều này mang lại sự tăng tốc gấp hai hoặc ba lần cho mỗi kỷ nguyên.

Chúng tôi đã bảo vệ khối mã này bằng các mệnh đề `try:` và `except:`; nếu nó hoạt động, chúng ta sẽ đạt được tốc độ xử lý nhanh hơn, nếu nó thất bại, sẽ không có gì xảy ra.

10.9.4 Sử dụng các mô hình CNN được huấn luyện trước

Tiếp theo, chúng tôi sẽ trình bày cách sử dụng mạng CNN được huấn luyện trước trên cơ sở dữ liệu `ImageNet` để phân loại ảnh tự nhiên, và minh họa cách chúng tôi tạo ra Hình 10.10.

Chúng tôi đã sao chép sáu hình ảnh JPEG từ một album ảnh kỹ thuật số vào thư mục `book_images`. Những hình ảnh này có sẵn trong phần dữ liệu của trang web www.statlearning.com . Trang web sách ISLP. Tải xuống `book_images.zip`; khi nhấp vào, nó sẽ tạo thư mục `book_images` .

Mạng nơ-ron được huấn luyện trước mà chúng ta sử dụng có tên là `resnet50`; chi tiết về thông số kỹ thuật có thể được tìm thấy trên web. Chúng ta sẽ đọc các hình ảnh và chuyển đổi chúng sang định dạng mảng mà phần mềm `torch` yêu cầu để phù hợp với các thông số kỹ thuật trong `resnet50`. Quá trình chuyển đổi bao gồm thay đổi kích thước, cắt xén và sau đó chuẩn hóa theo định nghĩa trước cho mỗi trong ba kênh. Bây giờ chúng ta sẽ đọc các hình ảnh và tiền xử lý chúng.

```
Trong [61]: resize = Resize((232,232))
crop = CenterCrop(224)
normalize = Normalize([0.485,0.456,0.406],
                      [0.229,0.224,0.225])
imgfiles = sorted([f for f in glob('book_images/*')])
imgs = torch.stack([torch.div(crop(resize(read_image(f))), 255)
                    for f in imgfiles])
imgs = normalize(imgs)
imgs.size()
```

```
Out[61]: torch.Size([6, 3, 224, 224])
```

Giờ chúng ta thiết lập mạng đã được huấn luyện với các trọng số mà chúng ta đã đọc trong mã. Khối 6. Mô hình này có 50 lớp, với độ phức tạp khá cao.

```
Trong [62]: resnet_model = resnet50(weights=ResNet50_Weights.DEFAULT)
tóm tắt(resnet_model,
        input_data=imgs,
        col_names=['input_size',
                  'output_size',
                  'num_params'])
```

Chúng tôi đặt chế độ thành `eval()` để đảm bảo mô hình sẵn sàng dự đoán trên dữ liệu mới.

```
Trong [63]: resnet_model.eval()
```

Quan sát kết quả đầu ra ở trên, chúng ta thấy rằng khi thiết lập `resnet_model`, các tác giả đã định nghĩa một `Bottleneck`, tương tự như mô-đun `BuildingBlock` của chúng ta .

Giờ chúng ta sẽ đưa sáu hình ảnh này vào mạng nơ-ron đã được huấn luyện.

```
Trong [64]: img_preds = resnet_model(imgs)
```

Hãy cùng xem xét xác suất dự đoán cho mỗi trong 3 lựa chọn hàng đầu.

Đầu tiên, chúng ta tính toán xác suất bằng cách áp dụng hàm softmax cho các logit trong `img_preds`. Lưu ý rằng chúng ta đã phải gọi phương thức `detach()` trên tensor `img_preds` để chuyển đổi nó thành mảng `ndarray` quen thuộc hơn.

```
Trong [65]: img_probs = np.exp(np.asarray(img_preds.detach()))
img_probs /= img_probs.sum(1)[:,None]
```

Để xem được nhãn lớp, chúng ta phải tải xuống tệp chỉ mục liên kết với `imagenet`.

27

```
Trong [66]: labs = json.load(open('imagenet_class_index.json'))
class_labels = pd.DataFrame([(int(k), v[1]) for k, v in
                             labs.items()],
                             columns=['idx', 'label'])
class_labels = class_labels.set_index('idx')
class_labels = class_labels.sort_index()
```

Bây giờ chúng ta sẽ xây dựng một khung dữ liệu cho mỗi tệp hình ảnh với các nhãn tương ứng. Ba xác suất cao nhất được ước tính bởi mô hình trên.

27Thông tin này có sẵn trên trang web của cuốn sách và tại s3.amazonaws.com/deep-learning-models/image-models/imagenet_class_index.json .

```
Trong [67]: đối với i, imgfile trong enumerate(imgfiles):
            img_df = class_labels.copy()
            img_df['prob'] = img_probs[i]
            img_df = img_df.sort_values(by='prob', ascending=False)[:3]
            print(f'Hình ảnh: {imgfile}')
            print(img_df.reset_index().drop(columns=['idx']))
```

```
Hình ảnh: book_images/Cape_Weaver.jpg
nhân          có thể
0          jacamar 0.287283
1 bee_eater 0.046768
2 bulbul 0.037507

Hình ảnh: book_images/Flamingo.jpg
xác suất nhân
0          chim hồng hạc 0.591761
1          cô mỏ thìa 0.012386
2 American_egret 0.002105

Hình ảnh: book_images/Hawk_Fountain.jpg
nhân          có thể
0 great_grey_owl 0.287959
1 con diều 0.039478
2          đài phun nước 0.029384

Hình ảnh: book_images/Hawk_cropped.jpg
nhân          có thể
0          diều 0.301830
1          jay 0.121674
2 con chim ác là 0.015513

Hình ảnh: book_images/Lhasa_Apso.jpg
nhân          có thể
0          Lhasa 0.151143
1          Chó Shih-Tzu 0.129850
2 Tibetan_terrier 0.102358

Hình ảnh: book_images/Sleeping_Cat.jpg
xác suất nhân
0 mèo vằn 0.173627
1 tiger_cat 0.110414
2 tấm thảm chùi chân 0.093447
```

Chúng ta thấy rằng mô hình khá tự tin về `Flamingo.jpg`, nhưng hơi...

Điều này ít đúng hơn đối với các hình ảnh khác.

Chúng ta kết thúc phần này với công đoạn dọn dẹp thường lệ.

```
Trong [68]: del(cifar_test,
              cifar_train,
              cifar_dm,
              cifar_module,
              cifar_logger,
              cifar_optimizer,
              cifar_trainer)
```

10.9.5 Phân loại tài liệu IMDB

Chúng tôi hiện đang triển khai các mô hình phân loại cảm xúc (Mục 10.4) trên

Bộ dữ liệu `IMDB`. Như đã đề cập ở khối mã 8 phía trên, chúng ta đang sử dụng phiên bản đã được xử lý trước của bộ dữ liệu `IMDB` có trong gói `keras`. Vì `keras` sử dụng

Sử dụng `TensorFlow`, một thư viện tensor và học sâu khác, chúng tôi đã chuyển đổi dữ liệu để phù hợp với `Torch`. Mã được sử dụng để chuyển đổi từ `Keras` có sẵn trong module `ISLP.torch._make_imdb`. Nó yêu cầu một số gói `Keras` để chạy. Dữ liệu này sử dụng một từ điển có kích thước `10.000`.

Chúng tôi đã lưu trữ ba dạng biểu diễn khác nhau của dữ liệu đánh giá cho bài thực hành này:

- `load_tensor()`, một phiên bản tensor thưa có thể sử dụng được bởi `torch`;
- `load_sparse()`, một phiên bản ma trận thưa có thể sử dụng được bởi `sklearn`, vì chúng ta sẽ so sánh với kết quả phù hợp bằng `lasso`;
- `load_sequential()`, một phiên bản được thêm phần đệm của chuỗi biểu diễn gốc, giới hạn ở `500` từ cuối cùng của mỗi bài đánh giá.

```
Trong [69]: (imdb_seq_train,
            imdb_seq_test) = load_sequential(root='data/IMDB') padded_sample
            = np.asarray(imdb_seq_train.tensors[0][0]) sample_review =
            padded_sample[padded_sample > 0][:12] sample_review[:12]
```

```
Out[69]: array([ 1,          14, 22, 65,          16, 43, 530, 973, 1622, 1385,
                458, 4468], dtype=int32)
```

Các tập dữ liệu `imdb_seq_train` và `imdb_seq_test` đều là các thể hiện của lớp `TensorDataset`. Các tensor được sử dụng để xây dựng chúng có thể được tìm thấy trong thuộc tính `tensors`, với tensor đầu tiên là các đặc trưng `X` và tensor thứ hai là kết quả `Y`. Chúng tôi đã lấy hàng đặc trưng đầu tiên và lưu trữ nó dưới dạng `padded_sample`. Trong quá trình tiền xử lý được sử dụng để tạo ra các dữ liệu này, các chuỗi được thêm các số 0 vào đầu nếu chúng không đủ dài, do đó chúng tôi loại bỏ phần đệm này bằng cách giới hạn chỉ những mục có `padded_sample > 0`.

Tiếp theo, chúng tôi cung cấp 12 từ đầu tiên của bài đánh giá mẫu.

Chúng ta có thể tìm thấy những từ này trong từ điển tra cứu của mô-đun `ISLP.torch.imdb`.

```
Trong [70]: lookup = load_lookup(root='data/IMDB')
            ''.join(lookup[i] for i in sample_review)
```

```
Ra[70]: "<BẮT ĐẦU> bộ phim này thật xuất sắc về dàn diễn viên, địa điểm quay phim, phong cảnh
        hướng câu chuyện của mọi người"
```

Đối với mô hình đầu tiên, chúng tôi đã tạo ra một đặc trưng nhị phân cho mỗi trong số `10.000` từ có thể có trong tập dữ liệu, với giá trị một trong mục `i, j` nếu từ `j` xuất hiện trong bài đánh giá `i`. Vì hầu hết các bài đánh giá khá ngắn, ma trận đặc trưng như vậy có hơn 98% là số không. Dữ liệu này được truy cập bằng cách sử dụng hàm `load_tensor()` từ thư viện `ISLP`.

```
Trong [71]: max_num_workers=10
            (imdb_train,
             imdb_test) = load_tensor(root='data/IMDB') imdb_dm =
            SimpleDataModule(imdb_train, imdb_test,

                               validation=2000,
                               num_workers=min(6, max_num_workers),
                               batch_size=512)
```

Chúng ta sẽ sử dụng mô hình hai lớp cho mô hình đầu tiên.

```
Trong [72]: class IMDBModel(nn.Module):

    def __init__(self, input_size):
        super(IMDBModel, self).__init__() self.dense1
        = nn.Linear(input_size, 16) self.activation = nn.ReLU()
        self.dense2 = nn.Linear(16, 16)
        self.output = nn.Linear(16, 1)

    def forward(self, x):
        val = x
        for _map in [self.dense1,
                      self.activation,
                      self.dense2,
                      self.activation,
                      self.output]: val
            = _map(val) return
        torch.flatten(val)
```

Bây giờ chúng ta khởi tạo mô hình và xem bản tóm tắt (không hiển thị).

```
Trong [73]: imdb_model = IMDBModel(imdb_test.tensors[0].size()[1])
summary(imdb_model,
        input_size=imdb_test.tensors[0].size(),
        col_names=['input_size',
                   'output_size',
                   'num_params'])
```

Chúng ta sẽ tiếp tục sử dụng tốc độ học nhỏ hơn cho dữ liệu này, do đó chúng ta truyền một trình tối ưu hóa cho `SimpleModule`. Vì các đánh giá được phân loại theo cảm xúc tích cực hoặc tiêu cực, chúng ta sử dụng `SimpleModule.binary_classification()`.²⁸

```
Trong [74]: imdb_optimizer = RMSprop(imdb_model.parameters(), lr=0.001) imdb_module =
SimpleModule.binary_classification( imdb_model, optimizer=imdb_optimizer)
```

Sau khi tải các tập dữ liệu vào một mô-đun dữ liệu và tạo một `SimpleModule`, Các bước còn lại thì quen thuộc.

```
Trong [75]: imdb_logger = CSVLogger('logs', name='IMDB') imdb_trainer =
Trainer(deterministic=True,
        max_epochs=30,
        logger=imdb_logger,
        callbacks=[ErrorTracker()])

imdb_trainer.fit(imdb_module,
                 datamodule=imdb_dm)
```

Đánh giá sai số kiểm tra cho thấy độ chính xác khoảng 86%.

```
Trong [76]: test_results = imdb_trainer.test(imdb_module, datamodule=imdb_dm)
kết quả kiểm tra
```

²⁸Việc chúng tôi sử dụng `binary_classification()` thay vì `classification()` là do một số điểm tinh tế trong cách hoạt động của `torchmetrics.Accuracy()`, cũng như kiểu dữ liệu của các mục tiêu.

```
Out[76]: [{'test_loss': 1.0863, 'test_accuracy': 0.8550}]
```

So sánh với Lasso

Giờ đây, chúng ta sẽ sử dụng mô hình hồi quy logistic lasso với hàm `LogisticRegression()` từ thư viện `sklearn`. Vì `sklearn` không nhận dạng được các tensor thừa của `torch`, chúng ta sẽ sử dụng một ma trận thừa mà `sklearn` nhận dạng được.

```
Trong [77]: ((X_train, Y_train),
            (X_hợp_lệ, Y_hợp_lệ),
            (X_test, Y_test)) = load_sparse(validation=2000, random_state=0,
                                           root='data/IMDB')
```

Tương tự như những gì chúng ta đã làm trong Phần 10.9.1, chúng ta xây dựng một dãy gồm 50 các giá trị cho tham số điều chỉnh lasso λ .

```
Trong [78]: lam_max = np.abs(X_train.T * (Y_train - Y_train.mean())).max()
lam_val = lam_max * np.exp(np.linspace(np.log(1), np.log(1e-4),
                                       50))
```

Với hàm `LogisticRegression()`, tham số điều chỉnh C được chỉ định là nghịch đảo của λ . Có một số thuật toán giải cho hồi quy logistic; ở đây chúng ta sử dụng `liblinear`, hoạt động tốt với định dạng đầu vào thừa.

```
Trong [79]: logit = LogisticRegression(penalty='l1',
                                     C=1/lam_max,
                                     solver='liblinear',
                                     warm_start=True,
                                     fit_intercept=True)
```

Quá trình xử lý 50 giá trị mất khoảng 40 giây.

```
Trong [80]: coefs = []
intercepts = []

for l in lam_val:
    logit.C = 1/l
    logit.fit(X_train, Y_train)
    coefs.append(logit.coef_.copy())
    intercepts.append(logit.intercept_)
```

Hệ số và các điểm cắt trục tung có một chiều không cần thiết, có thể được loại bỏ bằng hàm `np.squeeze()`.

```
Trong [81]: coefs = np.squeeze(coefs)
intercepts = np.squeeze(intercepts)
```

Bây giờ chúng ta sẽ lập biểu đồ để so sánh kết quả của mạng nơ-ron với thuật toán lasso.

```
Trong [82]: %%capture
fig, axes = subplots(1, 2, figsize=(16, 8), sharey=True) for ((X_, Y_),
data_, color) in

    zip([(X_train, Y_train),
        (X_hợp_lệ, Y_hợp_lệ),
        (X_test, Y_test)],
```



```
['Huấn luyện', 'Xác thực', 'Kiểm tra'], ['đen', 'đỏ',
'xanh dương']): linpred_ = X_ * coefs.T
+ intercepts[None,:] label_ = np.array(linpred_ > 0) accuracy_ =
np.array([np.mean(Y_ == 1) for 1 in label_.T])
axes[0].plot(-np.log(lam_val / X_train.shape[0]),
accuracy_,
'--',
color=color,
markersize=13,
linewidth=2,
label=data_)
axes[0].legend()
axes[0].set_xlabel(r'$-\log(\lambda)$', fontsize=20) axes[0].set_ylabel('Độ
chính xác', fontsize=20)
```

Hãy chú ý việc sử dụng `%%capture`, lệnh này sẽ ngăn hiển thị hình ảnh chưa hoàn thiện. Điều này rất hữu ích khi tạo một hình ảnh phức tạp, vì các bước có thể trải rộng trên hai hoặc nhiều ô. Bây giờ chúng ta thêm biểu đồ về độ chính xác của lasso và hiển thị hình ảnh đã hoàn thiện bằng cách đơn giản nhập tên của nó vào cuối ô.

`%%chiếm lấy`

```
Trong [83]: imdb_results = pd.read_csv(imdb_logger.experiment.metrics_file_path)
summary_plot(imdb_results, axes[1],

col='accuracy',
ylabel='Accuracy')
axes[1].set_xticks(np.linspace(0, 30, 7).astype(int)) axes[1].set_ylabel('Accuracy',
fontsize=20) axes[1].set_xlabel('Epoch', fontsize=20)
axes[1].set_ylim([0.5, 1]); axes[1].axhline(test_results[0]
['test_accuracy'],

color='blue',
linestyle='--',
linewidth=3)

quá sung
```

Từ các đồ thị, chúng ta thấy rằng độ chính xác của hồi quy logistic lasso đạt đỉnh ở khoảng 0,88, tương tự như đối với mạng nơ-ron. Cuối cùng, chúng ta kết thúc bằng công đoạn dọn dẹp.

```
Trong [84]: del(imdb_model,
imdb_trainer,
imdb_logger,
imdb_dm,
imdb_train,
imdb_test)
```

10.9.6 Mạng thần kinh hồi quy

Trong thí nghiệm này, chúng tôi sẽ tiến hành hiệu chỉnh các mô hình được minh họa trong Phần 10.5.

Mô hình tuần tự cho phân loại tài liệu

Ở đây, chúng tôi huấn luyện một mạng RNN LSTM đơn giản để dự đoán cảm xúc dựa trên dữ liệu đánh giá phim IMDB , như đã thảo luận trong Phần 10.5.1. Đối với mạng RNN, chúng tôi sử dụng...

Trình tự các từ trong một tài liệu, có tính đến thứ tự xuất hiện của chúng. Chúng tôi đã tải dữ liệu đã được xử lý trước ở đầu Mục 10.9.5. Một đoạn mã mô tả chi tiết quá trình xử lý trước có thể được tìm thấy trong thư viện ISLP . Đáng chú ý, vì hơn 90% tài liệu có ít hơn 500 từ, chúng tôi đã đặt độ dài tài liệu là 500 từ. Đối với các tài liệu dài hơn, chúng tôi đã sử dụng 500 từ cuối cùng, và đối với các tài liệu ngắn hơn, chúng tôi đã thêm khoảng trắng vào đầu tài liệu.

```
Trong [85]: imdb_seq_dm = SimpleDataModule(imdb_seq_train,
                                         imdb_seq_test,
                                         validation=2000,
                                         batch_size=300,
                                         num_workers=min(6, max_num_workers) )
```

Lớp đầu tiên của mạng RNN là lớp nhúng có kích thước 32, sẽ được học trong quá trình huấn luyện. Lớp này mã hóa one-hot mỗi tài liệu thành một ma trận có kích thước 500×10.003, sau đó ánh xạ 10.003 chiều này xuống còn 32.

29 Vì mỗi từ được biểu diễn bằng một số nguyên, điều này được thực hiện hiệu quả bằng cách tạo ra một ma trận nhúng có kích thước 10.003 × 32; mỗi trong số 500 số nguyên trong tài liệu sau đó được ánh xạ tới 32 số thực tương ứng bằng cách lập chỉ mục các hàng thích hợp của ma trận này.

Lớp thứ hai là một mạng LSTM với 32 đơn vị, và lớp đầu ra là một mạng logit đơn cho nhiệm vụ phân loại nhị phân. Ở dòng cuối cùng của phương thức forward() bên dưới, chúng ta lấy đầu ra 32 chiều cuối cùng của mạng LSTM và ánh xạ nó vào phản hồi của chúng ta.

```
Trong [86]: class LSTMModel(nn.Module): def
            __init__(self, input_size):
                super(LSTMModel, self).__init__() self.embedding
                = nn.Embedding(input_size, 32) self.lstm = nn.LSTM(input_size=32,
                                                                    hidden_size=32,
                                                                    batch_first=True)
                self.dense = nn.Linear(32, 1) def
            forward(self, x): val, (h_n,
                                c_n) = self.lstm(self.embedding(x)) return
            torch.flatten(self.dense(val[:,-1]))
```

Chúng ta khởi tạo và xem xét tóm tắt mô hình, sử dụng 10 tài liệu đầu tiên trong tập dữ liệu.

```
Trong [87]: lstm_model = LSTMModel(X_test.shape[-1]) summary(lstm_model,
                    input_data=imdb_seq_train.tensors[0][:10], col_names=['input_size',
                    'output_size', 'num_params'])
```

Out[87]: =====

Lớp (loại: chỉ số độ sâu)	Hình dạng đầu vào	Hình dạng đầu ra	Tham số #
Mô hình LSTM	[10, 500]	[10]	--

29Ba chiều bổ sung tương ứng với các mục không phải từ thường gặp trong đánh giá.

460 10. Học sâu

Nhúng: 1-1	[10, 500]	[10, 500, 32]	320.096
LSTM: 1-2	[10, 500, 32]	[10, 500, 32]	8.448
Tuyến tính: 1-3	[10, 32]	[10, 1]	33
=====			

Tổng số tham số: 328.577

Số tham số có thể huấn luyện: 328.577

Con số 10.003 bị ẩn đi trong phần tóm tắt, nhưng ta thấy nó trong số lượng tham số, vì 10.003 × 32 = 320.096.

```
Trong [88]: lstm_module = SimpleModule.binary_classification(lstm_model)
          lstm_logger = CSVLogger('logs', name='IMDB_LSTM')
```

```
Trong [89]: lstm_trainer = Trainer(deterministic=True,
                                max_epochs=20,
                                logger=lstm_logger,
                                callbacks=[ErrorTracker()])

lstm_trainer.fit(lstm_module,
                datamodule=imdb_seq_dm)
```

Phần còn lại hiện tương tự như các mạng khác mà chúng tôi đã huấn luyện. Chúng tôi theo dõi hiệu suất kiểm thử trong quá trình huấn luyện mạng và thấy rằng nó đạt độ chính xác 85%.

```
Trong [90]: lstm_trainer.test(lstm_module, datamodule=imdb_seq_dm)
```

Out[90]: {'test_loss': 0.8178, 'test_accuracy': 0.8476}

Chúng tôi một lần nữa trình bày quá trình học tập, tiếp theo là dọn dẹp.

```
Trong [91]: lstm_results = pd.read_csv(lstm_logger.experiment.metrics_file_path) fig, ax =
          subplots(1, 1, figsize=(6, 6)) summary_plot(lstm_results,

          riú,
          col='accuracy',
          ylabel='Accuracy')

ax.set_xticks(np.linspace(0, 20, 5).astype(int)) ax.set_ylabel('Độ
chính xác') ax.set_ylim([0.5, 1])
```

```
Trong [92]: del(lstm_model,
              lstm_trainer,
              lstm_logger,
              imdb_seq_dm,
              imdb_seq_train,
              imdb_seq_test)
```

Dự đoán chuỗi thời gian

Bây giờ chúng ta sẽ trình bày cách điều chỉnh các mô hình trong Phần 10.5.2 để dự đoán chuỗi thời gian. Trước tiên, chúng ta tải và chuẩn hóa dữ liệu.

```
Trong [93]: NYSE = load_data('NYSE') cols
          = ['DJ_return', 'log_volume', 'log_volatility']
X = pd.DataFrame(StandardScaler( with_mean=True,

                                with_std=True).fit_transform(NYSE[cols]),
               columns=NYSE[cols].columns,
               index=NYSE.index)
```

Tiếp theo, chúng ta thiết lập các phiên bản dữ liệu trễ, loại bỏ bất kỳ hàng nào có giá trị bị thiếu bằng phương thức `dropna()` .

```
Trong [94]: đối với độ trễ trong phạm vi (1, 6):
            cho col trong cols:

                newcol = np.zeros(X.shape[0]) * np.nan
                newcol[lag:] = X[col].values[:-lag]
                X.insert(len(X.columns), "{0}_{1}".format(col, lag), newcol)
X.insert(len(X.columns), 'train', NYSE['train'])
X = X.dropna()
```

Cuối cùng, chúng ta trích xuất phản hồi, chỉ số huấn luyện và loại bỏ `DJ_return` và `log_volatility` của ngày hiện tại để chỉ dự đoán từ dữ liệu của ngày hôm trước.

```
Trong [95]: Y, train = X['log_volume'], X['train']
X = X.drop(columns=['train'] + cols)
Cột X
```

```
Out[95]: Index(['DJ_return_1', 'log_volume_1', 'log_volatility_1',
               'DJ_return_2', 'log_volume_2', 'log_volatility_2',
               'DJ_return_3', 'log_volume_3', 'log_volatility_3',
               'DJ_return_4', 'log_volume_4', 'log_volatility_4',
               'DJ_return_5', 'log_volume_5', 'log_volatility_5'], dtype='object')
```

Đầu tiên, chúng tôi áp dụng một mô hình tuyến tính đơn giản và tính toán hệ số R^2 trên dữ liệu kiểm thử bằng phương pháp `score()` .

```
Trong [96]: M = LinearRegression()
M.fit(X[train], Y[train])
M.score(X[ train], Y[ train])
```

```
Out[96]: 0.4129
```

Chúng tôi hiệu chỉnh lại mô hình này, bao gồm cả biến yếu tố `day_of_week`. Đối với chuỗi dữ liệu phân loại trong `pandas`, chúng ta có thể tạo các chỉ báo bằng cách sử dụng phương thức `get_dummies()` .

```
Trong [97]: X_day = pd.merge(X,
                           pd.get_dummies(NYSE['day_of_week']), on='date')
```

Lưu ý rằng chúng ta không cần phải tạo lại mô hình hồi quy tuyến tính vì phương thức `fit()` của nó chấp nhận trực tiếp ma trận thiết kế và biến phản hồi.

```
Trong [98]: M.fit(X_day[train], Y[train])
M.score(X_day[ train], Y[ train])
```

```
Out[98]: 0.4595
```

Mô hình này đạt hệ số R^2 khoảng 46%.

Đề phù hợp với RNN, chúng ta phải định hình lại dữ liệu, vì nó sẽ yêu cầu 5 phiên bản trễ của mỗi đặc trưng như được chỉ định bởi đối số `input_shape` của lớp `nn.RNN()` bên dưới. Trước tiên, chúng ta đảm bảo các cột của khung dữ liệu sao cho ma trận được định hình lại sẽ có các biến được trễ chính xác. Chúng ta sử dụng phương thức `reindex()` để thực hiện điều này.